

同济大学

Tongji University

《操作系统课程设计》

项目报告

报告名称:	xv6 及 Labs 课程项目
课号:	42028703
学号:	2450333
姓名:	蒋昊沅
指导老师:	王冬青
日期:	2026.07-2026.08

目录

Lab0: Environment Setup	4
Lab1: Utilities	5
Boot xv6 (easy)	6
sleep (easy)	6
sixfive(moderate)	7
memdump(easy)	8
find(moderate)	9
find -exec(moderate)	10
实验结果	10
Lab2: System Calls	11
GDB and system calls(easy)	11
Sandbox a command(moderate)	12
Sandbox with allowed pathnames(easy)	14
Attack xv6(moderate)	15
实验结果	16
Lab3: Page Table	17
Inspect a user-process page table (easy)	17
Speed up system calls (easy)	19
Print a page table (easy)	20
Use superpages (moderate)/(hard)	21
实验结果	23
Lab4: Traps	23
RISC-V assembly (easy)	24
Backtrace (moderate)	25
Alarm (hard)	26
实验结果	28
Lab5: Copy-on-Write Fork	29
Copy-on-Write fork (hard)	30
实验结果	33
Lab6: Networking	34
Part One: NIC (moderate)	35
Part Two: UDP Receive (moderate)	37
实验结果	39
Lab7: Lock	40
Memory allocator (moderate)	40
Read-write lock (moderate)	43
实验结果	46

Lab8: File system	47
Large files (moderate)	47
Symbolic links (moderate)	50
实验结果	52
Lab9: mmap	53
Memory-mapped files (hard)	53
实验结果	57

Lab0: Environment Setup

对应源码分支: <https://github.com/JambitX11/xv6-labs-2025/tree/riscv>

课程实验网站: <https://pdos.csail.mit.edu/6.828/2025/>

本实验完成 xv6-labs-2025 所需开发环境的配置。整个运行环境由多层组成: Windows 提供宿主系统, WSL 中的 Ubuntu 提供 Linux 编译环境, RISC-V 交叉工具链负责生成目标程序, QEMU 模拟 RISC-V 计算机, 最后由 QEMU 启动 xv6。

Windows

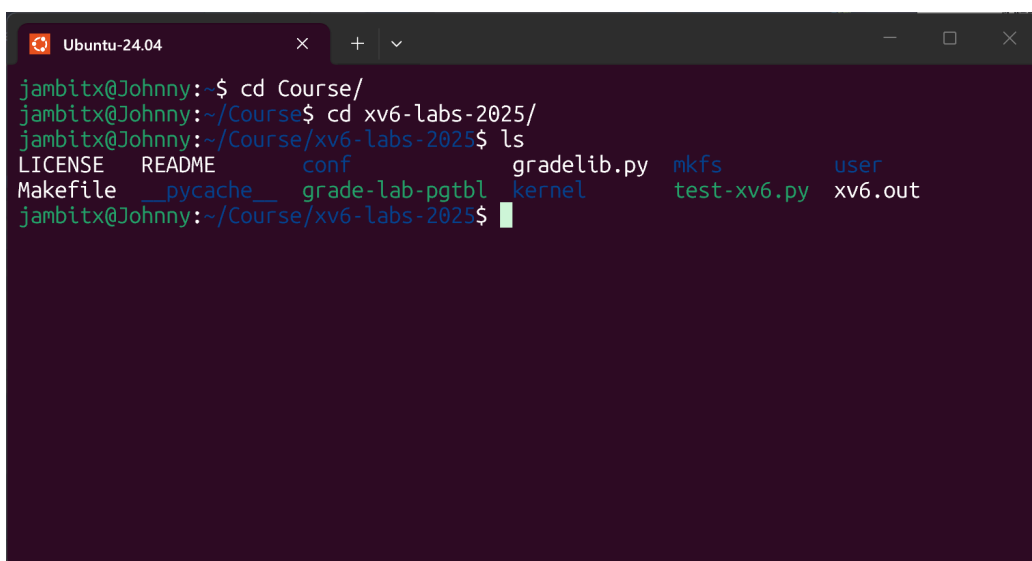
- > WSL 2 与 Ubuntu 24.04
- > RISC-V GCC、GDB、Make
- > QEMU 模拟 RISC-V 硬件
- > xv6 内核与 xv6 shell

实验目的

本实验的目标是建立一套可以编译、运行、调试和测试 xv6 的环境, 并确认 Git 分支能够正常与 GitHub 同步。完成环境配置后, 后续实验只需要切换到相应源码分支, 即可在 WSL 中编译并运行同一套 xv6 工程。

实验步骤

首先在 Windows 中启用 WSL 2, 并安装 Ubuntu 24.04。xv6 的源码虽然保存在 Windows 可以访问的仓库中, 但实际编译命令在 Ubuntu 终端内执行, 因此可以使用课程提供的 Linux 工具链和 Makefile。



```
jambitx@Johnny:~$ cd Course/
jambitx@Johnny:~/Course$ cd xv6-labs-2025/
jambitx@Johnny:~/Course/xv6-labs-2025$ ls
LICENSE  README      conf          gradelib.py  mkfs          user
Makefile __pycache__ grade-lab-pgtbl kernel        test-xv6.py  xv6.out
```

图 1 配置 WSL 2 与 Ubuntu 24.04

随后安装 Git、Make、GDB、QEMU 和 RISC-V 交叉编译工具链。这里的“交叉编译”表示编译器运行在 x86-64 的 WSL 环境中，但生成的是 RISC-V 指令，生成的程序不能由 Windows 或 WSL 直接执行，需要交给 QEMU 中的 RISC-V 虚拟机运行。

```
sudo apt update
sudo apt install git build-essential gdb-multiarch qemu-system-misc \
gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

安装完成后克隆实验仓库，并使用独立分支保存不同 Lab 的源码。util、syscall、pgtbl、traps 等分支保存相应实验实现，report 分支只保存 Typst 报告。这样可以在不混入报告文件的情况下比较各实验分支与 riscv 基线的代码差异。

最后在源码分支执行 `make qemu`。Makefile 会编译内核和用户程序、生成文件系统镜像，然后启动 QEMU。终端出现 xv6 的 `$` 提示符后，说明处理器模拟、内核启动和用户态 shell 均已正常工作。执行 `make grade` 则可运行课程提供的自动测试。

实验中遇到的问题和解决方法

最初使用的 WSL 发行版是 Ubuntu 22.04，其软件源提供的 QEMU 版本为 6.2，而 xv6-labs-2025 要求 QEMU 版本不低于 7.2。执行构建命令时，Makefile 的版本检查因此无法通过。

一种处理方法是自行编译新版 QEMU，但这会额外引入构建依赖和维护工作。本实验最终改用 Ubuntu 24.04，其软件源能够直接安装满足要求的 QEMU。重新安装 RISC-V GCC、GDB 和 QEMU 后，`make qemu` 与 `make grade` 均可正常运行。

实验心得

完成配置后，我明确了 xv6 并不是运行在 WSL 内核中的普通 Linux 程序。WSL 负责提供编译环境，QEMU 负责模拟硬件，xv6 才是运行在该硬件上的目标操作系统。后续在 xv6 shell 中执行的命令，也都是被编译进 `fs.img` 的 xv6 用户程序。

Lab1: Utilities

对应源码分支：<https://github.com/JambitX11/xv6-labs-2025/tree/util>

本实验通过编写若干 xv6 用户程序，熟悉命令行参数、系统调用、文件读取、内存解释、目录遍历和进程创建。新增用户程序的通用构建过程如下：

```
编写 user/*.c
-> 在 Makefile 的 UPROGS 中登记程序
-> RISC-V 工具链编译
```

```
-> 程序写入 fs.img  
-> 在 xv6 shell 中运行
```

Boot xv6 (easy)

实验目的

本任务用于确认 xv6 可以正常编译、启动并接受用户命令，同时区分 WSL shell 与 xv6 自身的 shell。后续编写的 sleep、sixfive 和 find 都需要先进入 xv6 才能运行。

实验步骤

在 util 分支执行 make qemu。该命令先编译 xv6 内核及用户程序，再生成文件系统镜像并启动 QEMU。启动日志末尾出现 init: starting sh 和 \$ 后，当前终端已经进入 xv6 shell。此时运行 ls，看到的是 xv6 文件系统的内容，而不是 WSL 目录。

本任务没有修改 xv6 源码，主要用于验证工具链、QEMU 和文件系统镜像的构建流程。

实验中遇到的问题和解决方法

初次启动时，Ubuntu 22.04 中的 QEMU 版本低于课程要求。将 WSL 环境更换为 Ubuntu 24.04 并重新安装依赖后，版本检查通过。另一个容易混淆的问题是两个 shell 都使用 \$ 作为提示符，可以通过 ls 的输出判断当前位于 WSL 还是 xv6。

实验心得

用户程序不会因为源文件出现在 user/ 目录中就自动出现在 xv6。程序还需要经过交叉编译并写入 fs.img，之后才能由 xv6 shell 查找和执行。

sleep (easy)

实验目的

本任务实现 sleep ticks 命令。用户输入一个 tick 数后，程序调用 xv6 已有的 pause() 系统调用，使当前进程等待相应时间。该任务用于熟悉用户程序如何接收参数并调用内核服务。

实验步骤

shell 执行 sleep 10 时，字符串 "sleep" 和 "10" 分别进入 argv[0] 与 argv[1]。user/sleep.c 先检查参数数量，再用 atoi() 将 "10" 转换为整数，随后调用 pause(10)。等待结束后，程序通过 exit(0) 返回 shell。

```
sleep 10
-> 从 argv[1] 取得字符串 "10"
-> atoi() 转换为整数 10
-> pause(10) 进入内核等待
-> 等待结束并退出
```

该任务修改了 `user/sleep.c` 和 `Makefile`。前者负责参数检查与系统调用，后者将 `$U/_sleep` 加入 `UPROGS`，使程序进入 `xv6` 文件系统镜像。

实验中遇到的问题和解决方法

程序必须检查 `argc`。如果缺少 `tick` 参数，继续读取 `argv[1]` 会访问不存在的参数，因此程序应输出用法并返回非零状态。新增源码后还需要修改 `UPROGS`；遗漏该步骤时，程序虽然可能已被编译，但 `xv6 shell` 中无法执行对应命令。

实验心得

`sleep` 的实现很短，但包含了用户命令的完整路径：`shell` 组织参数，用户程序解析参数，系统调用进入内核，内核完成等待后再返回用户态。

sixfive(moderate)

实验目的

本任务实现文本扫描程序 `sixfive`，读取一个或多个文件，找出其中能被 5 或 6 整除的独立十进制整数。题目要求区分“数字字符”和“完整整数”，例如 `/6`，中的 6 应被识别，而 `xv6` 中的 6 不应被单独取出。

实验步骤

`user/sixfive.c` 使用 `open()` 打开文件，并通过 `read()` 每次读取一个字符。程序不能只判断字符是不是数字，还要记住前面的输入处于什么状态，因此设置了三个状态：

<code>SEP_STATE</code>	当前位于合法分隔符之后，可以开始读取整数
<code>NUMBER_STATE</code>	当前正在读取一个合法整数
<code>INVALID_STATE</code>	当前处于字母等非法片段中，忽略到下一个分隔符

例如读取 `xv6 /18`，时，`x` 使程序进入 `INVALID_STATE`，后面的 6 因此不会被识别；读到空格和 `/` 后状态回到 `SEP_STATE`，接下来的 1 和 8 组成整数 18，逗号使该整数结束，程序再判断 18 是否能被 5 或 6 整除。

文件结束时没有实际分隔符，因此循环结束后还需要检查一次：如果状态仍为 `NUMBER_STATE`，说明文件最后一个整数尚未处理。主函数则依次打开、扫描并关闭每个输入文件。

该任务主要修改 `user/sixfive.c` 和 `Makefile`。`sixfive.c` 完成逐字符读取、状态转换、整数累积和整除判断，`Makefile` 负责把程序加入文件系统镜像。

实验中遇到的问题和解决方法

最初容易将规则写成“遇到非数字就结束当前数字”，这样会错误提取 `xv6` 中的 6。加入 `INVALID_STATE` 后，数字只有在合法分隔符之后出现时才能开始，问题得到解决。另一个边界是文件末尾，若不进行循环后的补充检查，位于文件最后且没有换行符的整数会被遗漏。

实验心得

该任务真正需要处理的是输入边界。状态机保存了当前字符之前的上下文，使程序能够根据同一个数字字符所在的位置作出不同判断。

memdump(easy)

实验目的

本任务实现 `memdump(fmt, data)`，按照格式字符串解释一段原始内存。它用于理解 C 语言中的类型转换：内存中保存的只是字节，程序按照 `int`、字符、指针或字符串读取时，这些字节才获得相应含义。

实验步骤

`user/memdump.c` 将 `data` 作为按字节移动的指针。每读取一个格式字符，就按指定类型解释当前位置，并按照该类型所占字节数移动 `data`。例如 `i` 读取 4 字节整数，`h` 读取 2 字节短整数，`c` 读取 1 字节字符，`p` 读取一个 64 位值。

`s` 与 `S` 的区别最能说明指针的作用：

```
S: data -> ['h']['e']['l']['l']['o']['\0']
```

`data` 指向的当前位置就是字符串内容

```
s: data -> [一个地址] -> ['h']['e']['l']['l']['o']['\0']
```

`data` 指向的位置保存的是字符串地址，需要先读取该地址

因此，处理 `S` 后要越过字符串本身及结尾的 `\0`，处理 `s` 后只需要越过一个指针的大小。该任务的实现集中在 `user/memdump.c`，`Makefile` 负责将对应测试程序编入 `xv6` 文件系统。

实验中遇到的问题和解决方法

实现时容易混淆“字符串内容”和“指向字符串的指针”。若把 `s` 按 `S` 处理，打印函数会把指针本身的字节当成字符；反过来处理也会把字符串开头的字节误认为地址。

另一个问题是读取后必须正确移动 `data`，偏移量错误会使后续所有字段从错误位置开始解释。

实验心得

`memdump` 说明类型信息并不存放在原始内存中，而是由读取内存的代码决定。这一认识也适用于后续实验中的 `trapframe`、页表项和文件系统结构体。

find(moderate)

实验目的

本任务实现简化版 `find`，从指定路径开始递归搜索名称匹配的文件或目录，并输出完整路径。该任务用于理解 `xv6` 如何表示目录，以及递归遍历如何通过文件系统接口完成。

实验步骤

`xv6` 将目录也表示为文件，目录内容由连续的 `struct dirent` 组成。`user/find.c` 先通过 `open()` 打开当前路径，再用 `fstat()` 判断它是普通文件还是目录。如果当前路径最后一段名称与目标相同，就记录一次匹配；如果它还是目录，则继续读取其中的目录项。

打开当前路径

- > `fstat()` 判断类型
- > 比较路径最后一段名称
- > 若为普通文件，结束当前分支
- > 若为目录，逐项读取并拼接子路径
- > 对每个子路径递归执行相同过程

目录项只保存短文件名，因此程序需要把父路径、斜杠和目录项名称拼接为完整子路径。名称比较时只取路径最后一段，例如 `./a/aa/b` 的匹配名称是 `b`。

该任务修改 `user/find.c` 和 `Makefile`。`find.c` 负责路径匹配、目录项读取、子路径构造与递归调用，`Makefile` 将 `$U/_find` 加入用户程序列表。

实验中遇到的问题和解决方法

遍历目录时必须跳过 `.` 和 `..`。前者指向当前目录，后者指向父目录，如果递归进入它们，程序会在已有目录之间反复循环。路径拼接前还要检查缓冲区剩余空间，并为目录项名称补上字符串结尾，避免路径越界或比较错误。

实验心得

实现 `find` 后，目录不再只是抽象的“文件夹”。程序实际读取一个个目录项，再根据 `inode` 类型决定是否继续递归，完整路径则由程序在遍历过程中逐层构造。

find -exec(moderate)

实验目的

本任务在 `find` 的基础上增加 `-exec`。找到匹配路径后，程序可以执行用户指定的命令，并把该路径追加到命令参数末尾。例如 `find . wc -exec echo hi` 找到 `./wc` 后，实际执行 `echo hi ./wc`。

实验步骤

实现仍位于 `user/find.c`。程序解析 `-exec` 后保存待执行命令及其参数；每次找到匹配项时，复制这些参数，把匹配路径追加为最后一个参数，并补上 `exec()` 要求的空指针结尾。

`find` 不能直接调用 `exec()`，因为 `exec()` 会用新程序替换当前进程。若直接替换，目录搜索会在第一次匹配后停止。因此每次匹配都采用以下过程：

```
find 找到匹配路径
-> fork() 创建子进程
-> 子进程 exec() 执行指定命令
-> 父进程 wait() 等待
-> find 继续遍历后续目录
```

为了让子目录中的匹配项也执行命令，递归调用 `find()` 时还需要继续传递 `-exec` 的模式、命令参数和参数数量。

实验中遇到的问题和解决方法

最初版本只实现了路径打印，基础递归搜索已经通过，但 `exec`, `recursive find` 测试得到 0 次输出，预期为 3 次。检查后确认，匹配处理函数没有构造 `exec()` 参数，也没有创建子进程。补充 `fork()`、`exec()` 和 `wait()` 后，每个匹配路径都能由独立子进程处理，递归测试通过。

实验心得

`find -exec` 将目录遍历和命令执行组合起来。`fork()` 保留继续搜索的父进程，`exec()` 只替换负责处理当前匹配项的子进程，`wait()` 则保证父进程在子进程结束后继续有序遍历。

实验结果

完成本 Lab 后，在 `util` 分支运行：

```
$ make grade
```

也可以使用 `GRADEFLAGS` 分别检查 `sleep`、`sixfive`、`memdump`、`find` 和 `exec`。各项测试最终均通过。由于实验完成时没有保留最终评分截图，报告中暂留如下位置：

待补充： 补充 `util` 分支最终 `make grade` 截图，建议保存为 `reports/assets/util/grade.png`。

本实验完成了 `xv6` 用户程序从源码、构建到运行的完整过程。几个任务分别使用了系统调用、状态机、指针类型转换、目录递归和 `fork-exec-wait`，这些机制也构成后续内核实验所使用的基础接口。

Lab2: System Calls

对应源码分支：<https://github.com/JambitX11/xv6-labs-2025/tree/syscall>

本实验围绕 `xv6` 的系统调用路径展开。用户程序没有权限直接操作内核数据，只能通过 `ecall` 请求内核完成工作。实验先用 GDB 观察一次请求如何到达内核，再在统一分发位置加入进程级拦截规则，最后通过未清零物理页实验观察内存复用带来的信息泄漏。

用户程序调用函数

```
-> 用户态 stub 把系统调用号放入 a7
-> ecall 进入内核
-> usertrap() 保存现场
-> syscall() 根据编号选择 sys_* 函数
-> 返回值写入 a0
-> 恢复用户程序
```

GDB and system calls(easy)

实验目的

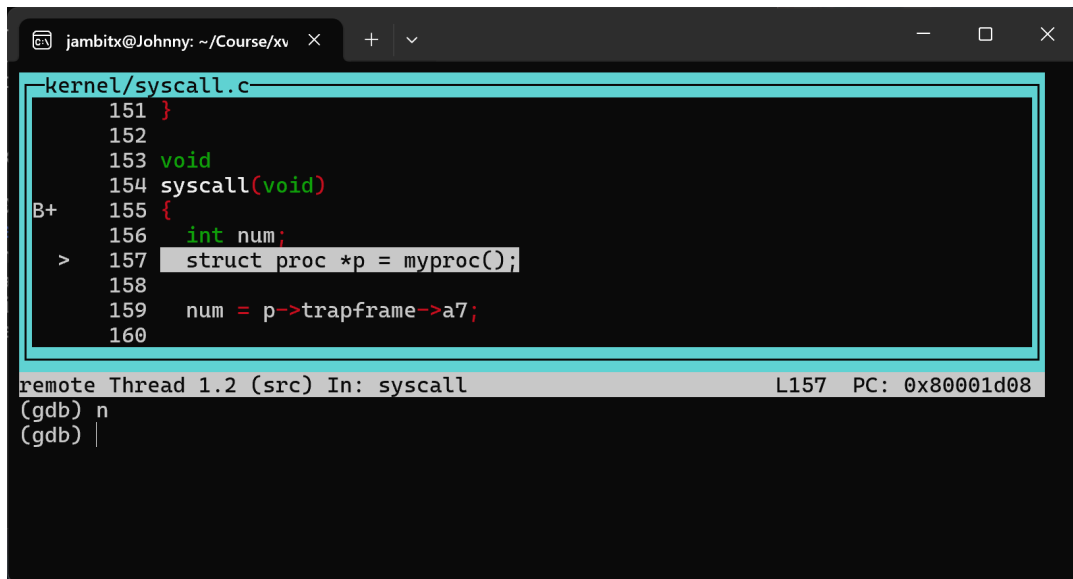
本任务使用 GDB 跟踪系统调用，确认用户态 `ecall`、内核 `trap` 入口、`trapframe` 和 `syscall()` 之间的关系。观察的重点是：系统调用编号保存在哪里，内核如何知道当前进程请求了什么服务，以及错误的内存访问会留下哪些异常信息。

实验步骤

首先执行 `make qemu-gdb`，让 QEMU 启动后等待调试器；随后使用 `gdb-multiarch kernel/kernel` 加载内核符号并连接 QEMU。在 `syscall()` 设置断点后继续运行，当用户程序发起系统调用时，GDB 会停在内核的系统调用分发函数中。

此时先单步执行到 `p = myproc()` 之后，再查看 `p->trapframe->a7`。a7 中保存系统调用号，a0、a1 等位置保存参数。执行 `backtrace` 可以看到 `syscall()` 是由 `usertrap()` 调用的；检查 `sstatus` 的 SPP 位，则可以判断 trap 发生前处理器处于用户态。

题目还通过故意访问地址 0 触发 load page fault。异常发生后，`scause` 说明异常类型，`stval` 保存出错的虚拟地址，`sepc` 指向触发异常的指令。将 `sepc` 与 `kernel/kernel.asm` 对照，可以定位具体的加载指令和目标寄存器。答案记录在 `answers-syscall.txt` 中。



The image shows a GDB window with the file `kernel/syscall.c` open. The code is as follows:

```
151 }
152
153 void
154 syscall(void)
155 {
156     int num;
157     struct proc *p = myproc();
158
159     num = p->trapframe->a7;
160 }
```

The GDB interface shows the current thread is `Thread 1.2 (src) In: syscall` at `L157` with `PC: 0x80001d08`. The GDB prompt is `(gdb) n`.

图 2 使用 GDB 观察 xv6 系统调用路径

该任务主要涉及 `answers-syscall.txt`，没有新增内核功能。`kernel/syscall.c`、`kernel/trap.c`、`kernel/proc.h` 和汇编输出文件用于调试观察。

实验中遇到的问题和解决方法

在断点刚进入 `syscall()` 时，局部变量 `p` 可能尚未赋值。若立即执行 `p->trapframe`，GDB 会显示无效地址。单步越过 `p = myproc()` 后再读取即可。连接 GDB 后若看不到函数名，则需要确认加载的是带符号的 `kernel/kernel`，而不是只连接 QEMU 后直接查看地址。

实验心得

系统调用表面上像一次函数调用，实际包含用户态到内核态的权限级切换。`trapframe` 保存了进入内核前的寄存器状态，所以 `syscall()` 可以从中读取编号和参数，也能把返回值放回用户程序将要恢复的 `a0`。

Sandbox a command(moderate)

实验目的

本任务新增 `interpose(mask, path)` 系统调用和用户程序 `sandbox`。它允许一个进程把指定系统调用设为禁止状态，并让该限制在执行其他程序后继续生效。目标命令一旦请求被禁止的系统调用，内核直接返回 `-1`。

`mask` 可以理解为一排开关。第 `n` 位对应第 `n` 号系统调用：该位为 `1` 表示禁止，为 `0` 表示允许。使用一个整数就能同时保存多项限制。

实验步骤

首先在 `kernel/proc.h` 的 `struct proc` 中增加 `syscall_mask` 和允许路径。限制必须保存在进程结构中，因为用户程序执行 `exec()` 后，原来的用户态变量会被新程序替换，而 `struct proc` 仍然属于同一个进程。

随后在 `kernel/syscall.h` 中分配 `interpose` 的系统调用号，在 `kernel/syscall.c` 中登记对应的内核处理函数，在 `user/user.h` 与 `user/usys.pl` 中加入用户态声明和 `stub`。`kernel/sysproc.c` 中的 `sys_interpose()` 读取 `mask` 与路径参数，并保存到当前进程。

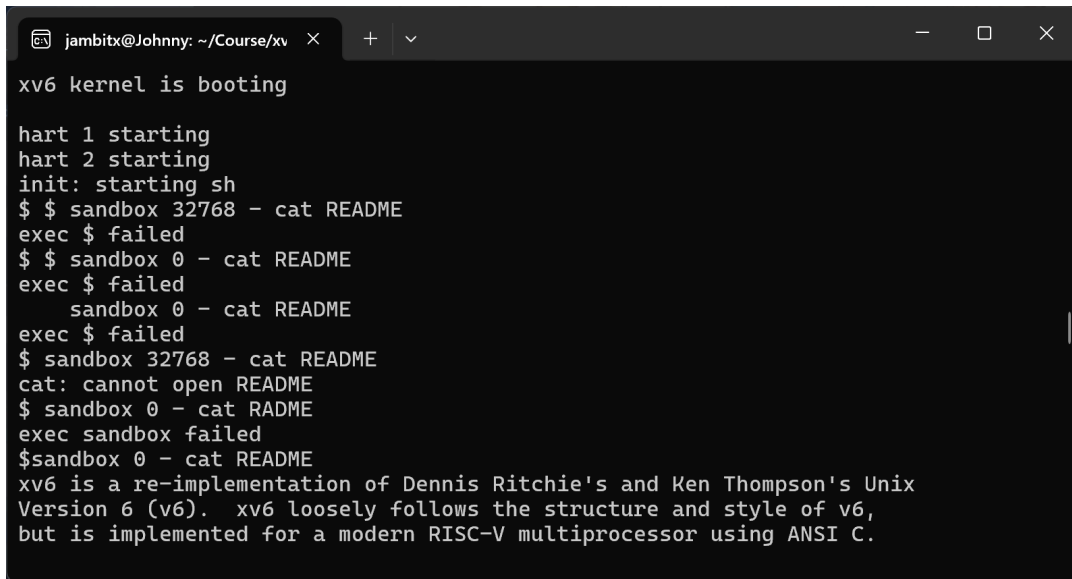
统一检查放在 `kernel/syscall.c` 的 `syscall()` 中。该函数原本根据 `a7` 中的编号直接调用 `syscalls[num]`；修改后，它先检查 `mask` 的第 `num` 位。若该位为 `1` 且不存在路径例外，就不再调用实际处理函数，而是把 `-1` 写入 `trapframe` 的 `a0`。

用户态 `user/sandbox.c` 负责启动受限命令。它先创建子进程，在子进程中调用 `interpose()` 设置限制，再通过 `exec()` 运行目标程序；父进程等待目标程序结束。

sandbox 启动目标命令

```
-> fork() 创建子进程
-> 子进程调用 interpose() 保存 mask
-> 子进程 exec() 运行目标程序
-> 目标程序每次系统调用都经过 syscall()
-> 命中 mask 时返回 -1
```

`kernel/proc.c` 的 `kfork()` 还需要把 `mask` 和路径复制给子进程。这样受限程序再次 `fork()` 时，新创建的进程也会继承相同规则。`Makefile` 则把 `sandbox` 加入 `xv6` 文件系统。

A terminal window titled 'jambitx@Johnny: ~/Course/xv' showing the output of the xv6 kernel booting. The output includes messages for hart 1 and 2 starting, init starting sh, and several attempts to run 'cat README' in a sandbox, all of which failed. The final message states that xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix Version 6 (v6), loosely following its structure and style, but implemented for a modern RISC-V multiprocessor using ANSI C.

```
jambitx@Johnny: ~/Course/xv
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ $ sandbox 32768 - cat README
exec $ failed
$ $ sandbox 0 - cat README
exec $ failed
    sandbox 0 - cat README
exec $ failed
$ sandbox 32768 - cat README
cat: cannot open README
$ sandbox 0 - cat RADME
exec sandbox failed
$ sandbox 0 - cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
```

图 3 Sandbox a command 任务测试结果

实验中遇到的问题和解决方法

如果限制只保存在 `sandbox.c` 的普通变量中, `exec()` 后这些变量会随原地址空间一起消失, 内核也无法在系统调用入口读取它们。将状态放入 `struct proc` 后, 限制可以跨越 `exec()`。测试还要求限制能够传递给后续子进程, 因此 `kfork()` 中必须显式复制相关字段。

实验心得

所有系统调用都经过 `syscall()`, 因此在这个统一入口检查 `mask`, 可以覆盖进程可能调用的全部内核服务。沙箱功能并不需要修改每一个 `sys_*` 函数, 关键是找到共同的分发位置, 并把策略保存在生命周期合适的进程结构中。

Sandbox with allowed pathnames(easy)

实验目的

本任务在系统调用屏蔽基础上增加路径例外。某个进程可能需要禁止大多数 `open` 或 `exec`, 但仍允许访问一个指定路径。内核在判断系统调用编号后, 还需要继续检查本次调用携带的路径参数。

实验步骤

路径例外只处理 `SYS_open` 和 `SYS_exec`, 因为这两个系统调用的第 0 个参数都是路径字符串。检查过程位于 `kernel/syscall.c`: 先判断当前编号是否属于这两个系统调用, 再通过 `argstr()` 把用户地址空间中的路径复制到内核缓冲区, 最后与进程保存的 `allowed_path` 比较。

系统调用被 mask 命中

- > 是否为 open 或 exec
- > 否：拒绝
- > 是：复制路径到内核
- > 路径等于 allowed_path：放行
- > 路径不同：拒绝

内核不能只保存用户态字符串指针。该地址属于用户页表，用户程序可能修改其内容，进程执行 `exec()` 后原地址也可能失效。`sys_interpose()` 因此在设置策略时就把字符串复制到 `struct proc` 的固定数组中。

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6). xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu). The main purpose of xv6 is as a teaching
$ sandbox 32768 README grep xv6 x
grep: cannot open x
$ QEMU: Terminated
```

图 4 Allowed pathnames 任务测试结果

实验中遇到的问题和解决方法

实现时在内核中直接使用 `strcmp()`，编译器报告函数未声明。xv6 内核不链接宿主机的标准 C 库，只能使用内核已有的字符串函数。改用 `strncmp()` 并限制比较长度为 `MAXPATH` 后，编译通过。参数 `"-"` 表示没有允许路径，遇到该值时不应产生任何例外。

实验心得

权限判断除了检查“调用了哪个系统调用”，还可能需要检查“这次调用操作了什么对象”。路径来自用户地址空间，内核必须先安全复制再比较，不能直接信任用户传入的地址。

Attack xv6(moderate)

实验目的

本任务利用实验环境中故意保留的未清零物理页，恢复前一个进程写入的秘密字符串。它用于说明页表隔离只能阻止两个正在运行的进程直接访问对方地址；如果物理页回收后保留旧内容，后来的进程仍可能通过内存复用读到这些数据。

实验步骤

`user/secret.c` 申请内存页，在其中写入固定标记 `This may help.`，并在偏移 16 字节处写入秘密。进程退出后，虚拟地址空间被销毁，对应物理页回到空闲链表，但实验版本没有清除页中的字节。

`user/attack.c` 随后通过 `sbrk()` 申请 8 个页面。物理分配器可能把刚释放的旧页面分配给 `attack`。程序扫描整段新内存，先查找固定标记，找到后再读取偏移 16 字节处的字符串。

```
secret 获得物理页并写入数据
-> secret 退出
-> 物理页回到空闲链表，内容仍保留
-> attack 通过 sbrk() 申请页面
-> 分配器可能交回同一物理页
-> attack 搜索标记并读取秘密
```

为了避免把随机残留字节误认为秘密，`attack.c` 还检查结果是否为非空字母数字字符串，并确认存在字符串结束符。`Makefile` 负责把 `attack` 测试程序加入文件系统。

实验中遇到的问题和解决方法

空闲链表的分配顺序会影响 `attack` 是否立即得到目标页，因此测试可能连续运行 `attack`。程序不能只检查某个固定页面或固定地址，而要遍历整个申请区域。检查固定标记与字符串格式后，可以减少误判。

实验心得

该任务说明物理页的清理也是进程隔离的一部分。旧进程已经退出、原虚拟地址也已经失效，但页中的数据仍然存在。内核重新把该页交给用户进程前应清零，否则新的地址映射会暴露旧数据。

实验结果

完成本 Lab 后，在 `syscall` 分支运行：

```
$ make grade
```

`answers-syscall.txt`、`sandbox mask`、`fork` 继承、路径例外、`attack` 和 `time` 测试均通过。


```
jambitx@Johnny:~/Course/xv6-labs-2025$ make grade
== Test answers-syscall.txt ==
answers-syscall.txt: OK
== Test sandbox_mask ==
$ make qemu-gdb
sandbox_mask: OK (2.9s)
== Test sandbox_fork ==
$ make qemu-gdb
sandbox_fork: OK (2.1s)
== Test sandbox_path ==
$ make qemu-gdb
sandbox_path: OK (0.9s)
== Test sandbox_most ==
$ make qemu-gdb
sandbox_most: OK (1.8s)
== Test sandbox_minus ==
$ make qemu-gdb
sandbox_minus: OK (3.1s)
== Test attack ==
$ make qemu-gdb
attack: OK (2.9s)
== Test time ==
time: OK
Score: 45/45
```

图 5 System Calls Lab 的 make grade 测试结果

本实验从实际系统调用路径出发，在 `syscall()` 入口增加统一限制，并通过物理页残留观察内存生命周期问题。最终测试通过说明系统调用 `mask`、子进程继承、路径例外和攻击程序均按要求工作。

Lab3: Page Table

对应源码分支：<https://github.com/JambitX11/xv6-labs-2025/tree/pgtbl>

本实验研究 RISC-V Sv39 页表。用户程序使用的是虚拟地址，内存条上的实际位置则由物理地址表示。页表位于二者之间，既负责地址翻译，也通过权限位决定某个页面能否被用户读取、写入或执行。

用户程序给出虚拟地址

- > Sv39 页表逐级查找 PTE
- > PTE 给出物理页和访问权限
- > 处理器访问实际物理内存

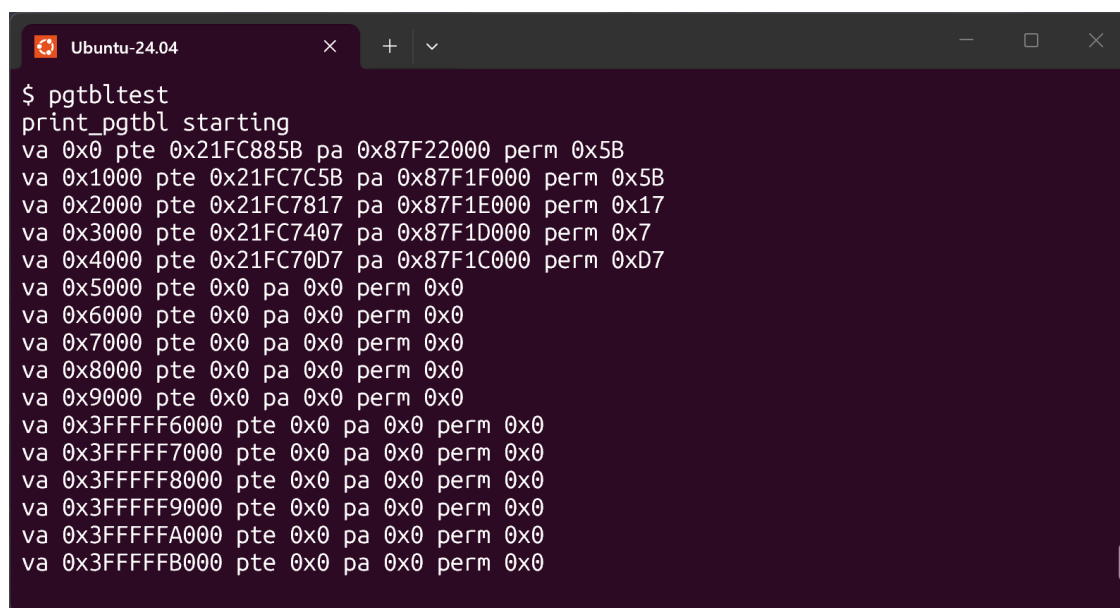
Inspect a user-process page table (easy)

实验目的

本任务运行 `pgtbltest`，观察当前用户进程低地址和最高地址附近的页表项，并解释输出中的虚拟地址、页表项、物理地址和权限。目标是先看懂 xv6 已有页表，再进行后续修改。

实验步骤

在 xv6 shell 中运行 `pgtbltest`。测试程序通过 `pgpte()` 查询前 10 个用户页面以及 `MAXVA` 之前的最后 10 个页面，因此输出同时覆盖普通程序区域和地址空间顶部的特殊映射。



```
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x2000 pte 0x21FC7817 pa 0x87F1E000 perm 0x17
va 0x3000 pte 0x21FC7407 pa 0x87F1D000 perm 0x7
va 0x4000 pte 0x21FC70D7 pa 0x87F1C000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
```

图 6 `pgtbltest` 输出中的用户进程页表项

每行输出可以按以下方式理解：

<code>va</code>	用户程序使用的虚拟地址
<code>pte</code>	页表项的完整 64 位数值
<code>pa</code>	从 <code>pte</code> 中取出的物理页地址
<code>perm</code>	<code>pte</code> 低位保存的权限标志

`PTE_V` 表示映射有效，`PTE_R`、`PTE_W` 和 `PTE_X` 分别表示可读、可写和可执行，`PTE_U` 表示用户态可以访问。代码页通常可读、可执行但不可写；数据页通常可读、可写但不可执行。若 `pte` 和 `perm` 都为 0，该虚拟页没有映射，访问时会产生缺页异常。

地址空间顶部的 `TRAPFRAME` 保存用户寄存器，`TRAMPOLINE` 保存用户态与内核态切换时执行的代码。它们出现在进程页表中，但没有 `PTE_U`，所以用户程序不能直接读写这些页面。

本任务最终修改 `answers-pgtbl.txt`，记录各页的逻辑用途和权限解释；`user/pgtbltest.c` 用于生成观察结果。

实验中遇到的问题和解决方法

分析时容易把 `pte` 直接当成物理地址。实际上一条 PTE 同时包含物理页号和低位标志，需要分别使用 `PTE2PA` 与 `PTE_FLAGS` 提取。另一个误区是认为相邻虚拟页必然对应相邻物理页。虚拟地址可以连续，但物理页由分配器独立选择，因此应逐项查看映射。

实验心得

页表项同时完成地址翻译和权限检查。用户程序看到的是连续虚拟空间，内核可以把它映射到分散的物理页，并对每一页设置不同权限。地址空间顶部的特殊页也说明，页面出现在用户页表中不等于用户态一定有权访问。

Speed up system calls (easy)

实验目的

本任务使用一页用户与内核共享的只读内存加速 `getpid()`。普通 `getpid()` 需要执行 `ecall`、进入内核、查找当前进程再返回；而 `pid` 在进程运行期间基本不变，可以由内核预先写入共享页，用户函数 `ugetpid()` 直接读取。

```
普通 getpid():
用户态 -> ecall -> usertrap() -> sys_getpid() -> 返回用户态

ugetpid():
用户态 -> 读取 USYSCALL 页面中的 pid
```

实验步骤

首先在 `kernel/memlayout.h` 中定义固定虚拟地址 `USYSCALL`，位置安排在 `TRAPFRAME` 下方，并定义只保存 `pid` 的 `struct usyscall`。固定地址使所有进程都能用同一个地址找到各自的共享页，但不同进程的该地址会映射到不同物理页。

`kernel/proc.h` 在进程结构中记录共享页的内核指针。`kernel/proc.c` 的 `allocproc()` 为新进程分配物理页并写入 `pid`；`proc_pagetable()` 将该页映射到用户虚拟地址 `USYSCALL`。权限只设置 `PTE_R | PTE_U`： `PTE_U` 允许用户访问，`PTE_R` 允许读取，没有 `PTE_W` 可以防止用户伪造 `pid`。

进程退出时，`proc_freepagetable()` 删除 `USYSCALL` 映射，`freeproc()` 再释放实际物理页。解除映射和释放物理内存由两个位置分别完成，避免同一页被释放两次。

最后在 `user/ulib.c` 中实现 `ugetpid()`，把固定地址解释为 `struct usyscall *` 并读取 `pid`。由于它只是普通内存读取，不需要新增系统调用号，也不执行 `ecall`。`user/user.h` 提供用户态声明。

实验中遇到的问题和解决方法

共享页必须设置 `PTE_U`，否则用户态读取会发生异常；同时不能设置 `PTE_W`，否则用户程序能够修改内核提供的数据。页面清理也需要明确责任：页表销毁函数只解除映射，`freeproc()` 才释放物理页。若两处都要求释放物理页，会产生重复释放。

实验心得

共享页适合保存由内核维护、用户频繁读取且不需要每次重新计算的数据。页表权限保证用户可以读到 `pid`，却不能修改它。这个优化减少了特权级切换，但仍保留内核对数据内容和页面生命周期的控制。

Print a page table (easy)

实验目的

本任务实现 `vmprint()`，按照 Sv39 的三级结构打印页表。页表原本只是内存中的多层数组，递归打印后可以直接看到哪些 PTE 指向下一级页表，哪些 PTE 已经是最终页面映射。

实验步骤

Sv39 把虚拟地址分成三级索引，每级索引选择一张页表中的一个 PTE。可以将它理解为三级目录：

```
level 2 顶层页表
-> level 1 中间页表
    -> level 0 底层页表
        -> 4KB 物理页
```

有效 PTE 有两种用途。若只设置 `PTE_V` 而没有读、写、执行权限，它保存的是下一级页表地址；若包含 `PTE_R`、`PTE_W` 或 `PTE_X`，它就是叶子 PTE，已经给出最终物理页面。

`kernel/vm.c` 中新增 `vmprint()` 和递归辅助函数。函数从 level 2 开始扫描当前页表的 512 个 PTE，跳过无效项，打印该项对应的虚拟地址前缀、PTE 和物理地址。遇到非叶子项时，把它指向的物理页当作下一张页表继续递归；遇到叶子项时只打印，不再向下。

每深入一级，输出前增加一组 `...`，因此缩进直接反映页表深度。`kernel/defs.h` 增加函数声明，系统调用 `kpgtbl()` 则调用 `vmprint()` 打印指定页表。

实验中遇到的问题和解决方法

如果把所有有效 PTE 都当成下一级页表，程序会把普通数据页中的字节继续解释为 PTE，导致错误输出或内核异常。因此递归前必须检查读、写、执行位，确认当前项是非

叶子项。测试还要求固定的缩进与地址格式，物理地址可以随运行变化，但虚拟地址和层级结构必须一致。

实验心得

打印结果把三级页表的查找过程显示成一棵树。非叶子 PTE 负责继续查找，叶子 PTE 才完成地址翻译。这个区别也是 superpage 的基础，因为叶子项并不一定只能出现在 level 0。

Use superpages (moderate)/(hard)

实验目的

本任务让 xv6 在合适的用户内存区域使用 2MB superpage。普通页为 4KB，映射 2MB 内存需要 512 个普通页和 512 个底层 PTE；superpage 使用一个 level-1 叶子 PTE 就能覆盖同样区域。

普通方式：512 个 4KB 物理页 + 512 个 level-0 叶子 PTE
superpage：1 个 2MB 连续物理块 + 1 个 level-1 叶子 PTE

这样可以减少页表项数量，并提高 TLB 对大块连续内存的覆盖范围。代价是分配、复制和释放都必须认识这种更大的页面。

实验步骤

第一部分修改 kernel/kalloc.c。普通 kalloc() 继续管理 4KB 页，新增 superalloc() 与 superfree() 管理 2MB 对齐的连续物理块。初始化时将一部分物理内存放入 superpage 专用空闲链表，并确保这些地址不会同时进入普通页链表。

第二部分修改 kernel/vm.c 的映射与地址翻译。新增的 level-1 页表遍历函数在中间层找到目标 PTE，映射函数检查虚拟地址和物理地址是否都按 2MB 对齐，然后直接设置一个带读写权限的 level-1 叶子项。

uvmmalloc() 负责用户调用 sbrk() 后的内存增长。当当前虚拟地址按 2MB 对齐、剩余增长空间至少为 2MB 时，它优先调用 superalloc()；首尾不足 2MB 或未对齐的部分仍使用普通 4KB 页。

sbrk() 扩展用户内存

- > 当前地址是否按 2MB 对齐
- > 剩余空间是否至少为 2MB
- > 都满足：分配并映射 superpage
- > 否则：继续分配普通 4KB 页

由于 level-1 现在也可能出现叶子项，原先默认叶子只在 level 0 的代码都需要调整。`walk()` 遇到中间层叶子时应停止下降；`walkaddr()` 计算物理地址时，要为 superpage 保留 21 位页内偏移，而普通页只保留 12 位偏移。

第三部分处理进程复制。`kernel/vm.c` 的 `uvmcopy()` 在 `fork()` 时检查父进程当前映射类型。普通页仍按 4KB 分配和复制；遇到 level-1 superpage 时，子进程分配新的 2MB 物理块，复制整块内容，并建立相同大小的映射。父子进程由此拥有内容相同但相互独立的物理内存。

第四部分处理释放。若要解除完整的 2MB 映射，可以清除 level-1 PTE 并调用 `superfree()`。若只释放其中最后 4KB，不能直接释放整个 superpage。实现通过 `demote_superpage()` 新建一张 level-0 页表，把原来的 2MB 映射改写为 512 个普通 4KB PTE，再按原有逻辑释放目标小页。

部分释放 2MB superpage

- > 保留原 2MB 物理内容
- > 建立 512 个 4KB PTE
- > 用 level-0 页表替换原 level-1 叶子
- > 只解除要求释放的 4KB 映射

`kernel/memlayout.h` 与 `kernel/riscv.h` 补充 superpage 大小、对齐和叶子判断相关定义，`kernel/defs.h` 增加分配及释放函数声明。主要行为集中在 `kernel/kalloc.c` 和 `kernel/vm.c`。

实验中遇到的问题和解决方法

第一次编译时，`issuperpage()` 在 `walksuper()` 定义之前调用它，编译器产生 `implicit declaration` 和 `conflicting types` 错误；`uvmcopy()` 中还保留了未使用的 `pte` 变量。将辅助函数声明移到调用之前，并删除未使用变量后，`kernel/vm.o` 可以正常编译。

随后 `superpg_fork` 报告 `sbrk failed`。该测试一次申请多个 2MB 页面，如果 superpage 空闲链表没有正确初始化，或预留的连续块数量不足，`superalloc()` 会过早返回空。重新划分普通页区域与 superpage 区域，并预留足够的 2MB 对齐块后，内存增长成功。

`superpg_free` 还要求只释放 superpage 尾部的 4KB，同时保留前面内容。直接清除 level-1 PTE 会让整个 2MB 映射消失，因此最终加入降级过程，把大页拆成普通页映射后再部分释放。

实验心得

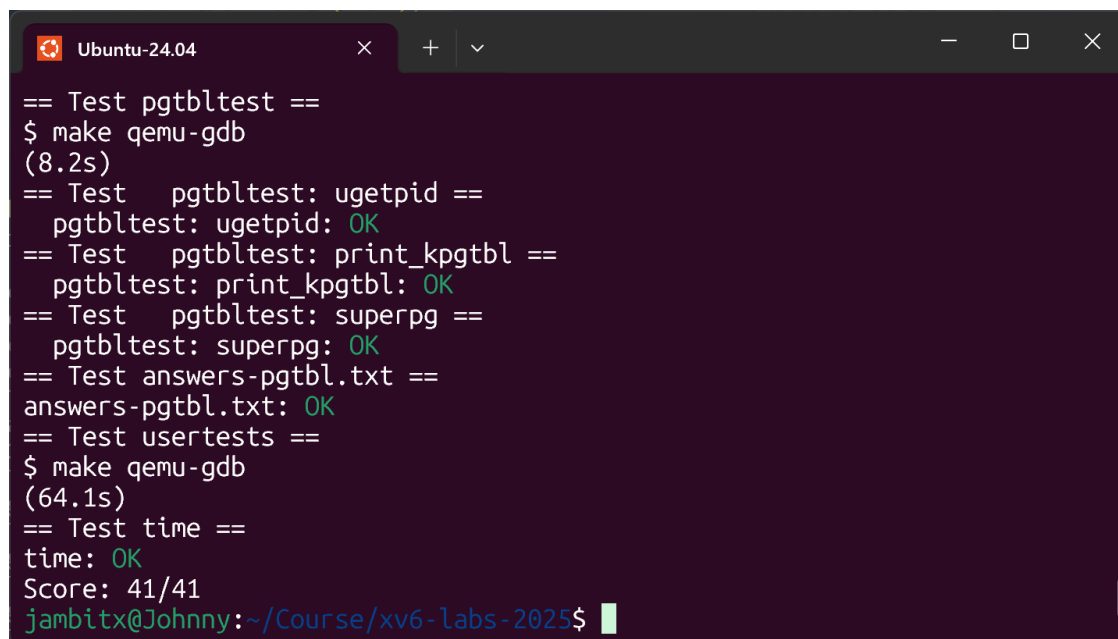
Superpage 并不是简单地把常量从 4KB 改为 2MB。叶子 PTE 所在层级发生变化后，物理分配、虚拟地址翻译、`fork()` 复制和内存释放都必须同步调整。部分释放尤其说明，大页提高了映射效率，但处理小粒度操作时需要先恢复为普通页结构。

实验结果

完成本 Lab 后，在 pgtbl 分支运行：

```
$ make grade
```

最终 pgtbltest、answers-pgtbl.txt、usertests 和 time 均通过，得分为 41/41。



```
== Test pgtbltest ==
$ make qemu-gdb
(8.2s)
== Test pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
== Test pgtbltest: print_kpgtbl ==
pgtbltest: print_kpgtbl: OK
== Test pgtbltest: superpg ==
pgtbltest: superpg: OK
== Test answers-pgtbl.txt ==
answers-pgtbl.txt: OK
== Test usertests ==
$ make qemu-gdb
(64.1s)
== Test time ==
time: OK
Score: 41/41
jambitx@Johnny:~/Course/xv6-labs-2025$
```

图 7 Page Table Lab 的 make grade 测试结果

本实验先读取现有页表，再增加共享只读页和页表打印，最后扩展到 2MB superpage。测试结果表明，页面权限、共享页生命周期、三级页表打印、superpage 映射、fork() 复制及部分释放均符合要求。

Lab4: Traps

对应源码分支：<https://github.com/JambitX11/xv6-labs-2025/tree/traps>

本实验围绕 xv6 中的 trap 机制展开。处理器在执行系统调用、发生异常或接收到中断时，需要暂停当前程序并进入内核；内核完成处理后，还要恢复用户程序原有的执行状态。实验先从 RISC-V 汇编入手，观察 C 函数调用在寄存器和指令层面的具体形式；随后利用内核栈帧实现调用栈回溯；最后通过修改时钟中断的处理过程，实现用户级定时告警。

从执行过程看，一次 trap 可以概括为：

用户程序正在运行

- > 系统调用、异常或中断发生
- > 保存用户寄存器并进入内核

```
-> 内核处理事件
-> 恢复寄存器并返回用户程序
```

三个任务分别研究这条路径所依赖的指令、内核栈和寄存器现场。

RISC-V assembly (easy)

实验目的

本任务要求阅读 `user/call.c` 及其编译生成的 `user/call.asm`，回答 `answers-traps.txt` 中有关函数参数、函数调用、返回地址、字节序和可变参数的问题。通过对照 C 代码与汇编代码，可以直接观察高级语言中的一次函数调用如何落实为寄存器赋值和跳转指令，并为后续分析 `trap` 保存的寄存器现场打下基础。

函数调用需要调用者和被调用者遵守同一套规则，否则被调用函数无法知道参数放在哪里。RISC-V 规定前八个参数依次放在 `a0` 至 `a7` 中，返回值通常也放在 `a0` 中。例如调用 `printf("%d %d\n", 12, 13)` 时，`a0` 保存格式字符串地址，`a1` 保存 12，`a2` 保存 13。

执行 `jal` 或 `jalr` 时，处理器一方面跳转到目标函数，另一方面把下一条指令地址写入 `ra`。`ra` 记录了函数执行完毕后应该回到哪里，其作用相当于保存返回位置。被调用函数最后执行 `ret`，处理器便根据 `ra` 回到调用者。该任务还涉及 `little-endian` 字节序，即一个多字节整数的低位字节存放在较低的内存地址中。

实验步骤

首先，在 `traps` 分支运行 `make fs.img`，由编译系统生成便于阅读的 `user/call.asm`。随后在该文件中定位 `<main>`、`<f>`、`<g>` 和 `<printf>`，检查调用 `printf` 前对参数寄存器的赋值。格式字符串地址位于 `a0`，两个整数参数依次位于 `a1` 和 `a2`，因此题目所问的整数 13 保存在 `a2` 中。`printf` 的地址由 `<printf>` 标签前的地址确定，调用结束后 `ra` 的值则是 `jal` 或 `jalr` 后一条指令的地址。

阅读汇编时还需以实际生成结果为准。由于 `f()` 和 `g()` 的逻辑较短，编译器可能将它们内联并提前计算 `f(8) + 1`，所以 `main()` 中不一定存在对这两个函数的跳转指令。这里没有根据 C 源码推测调用位置，而是通过 `call.asm` 判断编译器最终生成了哪些指令。

对于题目给出的字节序程序，将代码临时放入 `user/call.c` 的 `main()` 中并在 `xv6` 内运行。整数 `0x00646c72` 在 `little-endian` 机器中的内存字节为 `72 6c 64 00`，将其地址转换为字符串指针后得到 `"rld"`；十进制数 57616 以 `%x` 输出为 `e110`，因此完整输出为 `He110 World`。若处理器采用 `big-endian`，要得到相同字符串，应将整数调整为 `0x726c6400`，而 57616 不需要修改。

本任务最终只修改了 `answers-traps.txt`，其中记录各问题的分析结果。`user/call.asm` 是编译产物，`user/call.c` 仅用于临时验证题目中的程序，验证结束后恢复原内容。

实验中遇到的问题和解决方法

分析函数调用位置时，容易产生的误解是认为 C 源码中写出的函数一定会在汇编中对应一条 `call` 指令。实际检查发现，编译优化可能消除这类简单调用，因此答案必须以 `call.asm` 为依据。

对于 `printf("x=%d y=%d", 3)`，第二个 `%d` 没有对应的实参。`printf` 仍会按照调用约定读取下一个参数位置，但该位置中的内容并未由调用者设置，所以 `y` 的输出不是一个确定值。该现象源于 C 可变参数函数无法自动检查实参数数量，属于未定义行为，并非 `xv6` 的输出错误。

实验心得

本任务将函数调用约定从概念落实到了具体寄存器和指令上。参数、返回值和返回地址都由明确的寄存器承担，C 代码中的函数结构则可能被编译优化改变。阅读汇编时需要区分源程序表达的逻辑与处理器最终执行的指令，不能简单地在二者之间逐行对应。

Backtrace (moderate)

实验目的

本任务要求在内核中实现 `backtrace()`，打印到达当前位置之前经过的函数调用链。当内核发生 `panic` 或运行结果异常时，单独看到报错位置往往不足以判断原因；调用栈中的返回地址可以说明代码经过哪些函数到达当前状态，再借助 `addr2line` 将地址转换为源码位置，能够提高内核调试效率。

函数 A 调用函数 B，B 又调用函数 C 时，三次调用所需的局部数据和返回信息依次保存在内核栈中。每个函数占用的一段栈空间称为栈帧。`xv6` 使用 `s0` 作为 `frame pointer`，指向当前栈帧的固定位置；`fp - 8` 保存当前函数结束后要返回的地址，`fp - 16` 保存调用者的 `frame pointer`。栈帧之间因此形成了一条可以向上追踪的链。

当前 fp

fp - 8 -> 当前函数的返回地址

fp - 16 -> 上一层函数的 fp

|
-> 再读取上一层返回地址和 fp

`backtrace()` 所做的事情就是反复读取这两个位置。它不需要知道每个函数的名字，只先打印返回地址；随后再由 `addr2line` 根据 `kernel/kernel` 中的调试信息，把地址转换为源文件和行号。

实验步骤

首先修改 `kernel/riscv.h`，增加读取 `s0` 寄存器的内联函数 `r_fp()`，使 C 代码能够取得当前 frame pointer。随后在 `kernel/defs.h` 中声明 `backtrace()`，供其他内核文件调用。

核心实现位于 `kernel/printf.c`。函数先通过 `r_fp()` 获得当前栈帧地址，再使用 `PGROUNDUP(fp)` 和 `PGROUNDUP(fp)` 计算该地址所在内存页的上下边界。循环过程中，从 `fp - 8` 读取并打印当前函数保存的返回地址，再从 `fp - 16` 读取上一层 frame pointer。更新后的 `fp` 指向调用者的栈帧，下一轮循环便会打印调用者的返回地址。该过程持续到 `fp` 越过当前内核栈页。

为验证实现，在 `kernel/sysproc.c` 的 `sys_pause()` 中调用 `backtrace()`。运行 `bttest` 后，将打印出的地址输入 `addr2line -e kernel/kernel`，检查其能否依次对应到系统调用处理、系统调用分发和 `trap` 入口附近的源码。验证通过后，又在 `kernel/printf.c` 的 `panic()` 中调用 `backtrace()`，使内核发生 `panic` 时可以自动输出调用路径。

本任务涉及的文件及作用如下：

- `kernel/riscv.h`: 提供读取 frame pointer 的 `r_fp()`。
- `kernel/defs.h`: 加入 `backtrace()` 的函数声明。
- `kernel/printf.c`: 实现栈帧遍历，并在 `panic()` 中输出调用栈。
- `kernel/sysproc.c`: 在 `sys_pause()` 中触发 `backtrace`，供 `bttest` 验证。

实验中遇到的问题和解决方法

回溯过程不能只依靠“frame pointer 是否为 0”决定何时停止。内核栈只占一个页面，如果沿错误的地址继续读取，可能访问当前内核栈之外的内存，得到无意义的返回地址，甚至再次触发异常。为此，程序在循环前确定当前栈页边界，并在每次迭代前检查 `fp` 是否仍位于该页内。

控制台打印的是内核虚拟地址，无法直接看出对应的函数名和源文件。实验使用带有调试信息的 `kernel/kernel` 作为符号文件，通过 `addr2line` 完成地址解析，从而确认打印结果确实反映了 `bttest` 触发的调用链，并排除输出地址只是偶然处于合法范围内的情况。

实验心得

Backtrace 的实现代码较少，但它清楚地展示了函数调用在栈中的组织方式。每层函数调用都会留下返回地址和上一层栈帧的位置，调试器所展示的调用栈正是根据这些信息恢复出来的。同时，栈遍历必须受到有效内存范围的约束，能够读取一个地址不代表该地址属于当前调用链。

Alarm (hard)

实验目的

本任务为 `xv6` 增加用户级定时告警功能。用户程序调用 `sigalarm(interval, handler)` 后，进程每累计运行 `interval` 个时钟 `tick`，内核应安排其执行一次用户态 `handler`。处理函数调用 `sigreturn()` 后，程序继续执行被中断前的代码，并保持当时的寄存器值不变。

例如，用户程序执行 `sigalarm(2, periodic)` 后，每消耗两个时钟 `tick`，内核就应让它执行一次 `periodic()`。`periodic()` 不是一个永久替换原程序的新入口，它执行完 `sigreturn()` 后，原程序仍要从被中断的位置继续运行。

`trapframe` 可以理解为用户程序进入内核时留下的一份寄存器记录，其中既有程序计数器 `epc`，也有栈指针和通用寄存器。`Alarm` 使用这份记录改变程序下一步的执行位置，其完整过程如下：

用户程序正常执行

- > 时钟中断进入 `usertrap()`
- > 备份当前 `trapframe`
- > 将 `epc` 改成 `handler` 地址
- > 返回用户态并执行 `handler`
- > `handler` 调用 `sigreturn()`
- > 恢复原 `trapframe`
- > 从中断前的位置继续执行

实验步骤

第一步是为每个进程保存独立的 `alarm` 状态。在 `kernel/proc.h` 的 `struct proc` 中增加告警间隔、累计 `tick` 数、`handler` 地址、`handler` 是否正在执行的标志，以及一份备份 `trapframe`。这份备份保存“进入 `handler` 之前的程序状态”，不能直接使用当前 `trapframe` 代替，因为当前 `trapframe` 随后还要被修改为 `handler` 的入口状态。`kernel/proc.c` 负责在创建进程时初始化这些字段并分配备份空间，在进程释放时回收相应内存；创建子进程时，也需要保证子进程使用自己的计数和现场，不能与父进程共用备份。

第二步是接通两个系统调用。在 `kernel/syscall.h` 中分配 `sigalarm` 和 `sigreturn` 的系统调用号，在 `kernel/syscall.c` 中将编号映射到对应的内核处理函数；`user/user.h` 提供用户态声明，`user/usys.pl` 生成执行 `ecall` 的用户态入口。`kernel/sysproc.c` 中的 `sys_sigalarm()` 读取 `interval` 和 `handler` 参数，将其保存到当前进程，并重置计数状态；当 `interval` 为 0 时停止产生新的告警。`sys_sigreturn()` 将备份的 `trapframe` 恢复到当前 `trapframe`，并清除 `handler` 执行标志。

第三步是在 `kernel/trap.c` 的 `usertrap()` 中处理定时触发。`devintr()` 返回 2 表示本次 `trap` 来自时钟中断。此时，如果进程已经注册 `alarm` 且当前没有执行 `handler`，就将累计 `tick` 加一。达到 `interval` 后，内核复制完整 `trapframe`，清零计数并设置

执行标志，然后把当前 `trapframe->epc` 改为用户提供的 `handler` 地址。内核随后沿原有返回路径进入用户态，处理器便从 `handler` 开始执行。

`handler` 最终调用 `sigreturn()` 再次进入内核。恢复原 `trapframe` 后，原来的 `epc`、通用寄存器和栈指针重新生效，因此系统调用返回时会回到被时钟中断打断的位置。为避免系统调用分发代码把返回值覆盖到 `a0`，`sys_sigreturn()` 还需返回原现场中保存的 `a0`。

最后在 `Makefile` 的 `UPROGS` 中加入 `$U/_alarmtest`，使测试程序被写入 `xv6` 文件系统。本任务修改的文件包括 `kernel/proc.h`、`kernel/proc.c`、`kernel/syscall.h`、`kernel/syscall.c`、`kernel/sysproc.c`、`kernel/trap.c`、`user/user.h`、`user/usys.pl` 和 `Makefile`。这些改动分别完成进程状态保存、系统调用接入、时钟中断触发、用户态接口和测试程序构建。

实验中遇到的问题和解决方法

在 `Makefile` 中加入 `alarmtest` 后，首次编译出现 `recipe commences before first target`。检查后确认，该错误来自 `UPROGS` 列表的续行格式被破坏，与 C 代码无关。修正新增行的缩进和前一行末尾的反斜杠，并重新将 `$U/_alarmtest` 放入原有变量定义后，编译恢复正常。

`Alarm` 功能还需要处理两处容易被忽略的状态问题。其一，若 `handler` 执行期间继续累计并再次触发告警，第二次保存会覆盖第一次保存的现场，之后无法正确返回。因此使用进程内的执行标志阻止 `handler` 重入，并在 `sigreturn()` 中解除该标志。其二，普通系统调用会把返回值写入用户寄存器 `a0`。若 `sigreturn()` 直接返回固定值，恢复后的 `a0` 会再次被覆盖，导致 `alarmtest` 的寄存器保持测试失败。解决方法是在恢复 `trapframe` 的同时保留原 `a0`，并将其作为 `sys_sigreturn()` 的返回值。

实验心得

`Alarm` 实现中的主要难点是控制流切换前后的现场管理，`tick` 计数只负责确定触发时机。将 `epc` 改为 `handler` 地址只能让程序跳转过去；只有完整保存并恢复 `trapframe`，用户程序才能在 `handler` 结束后继续原来的计算。重入保护和 `a0` 恢复也说明，修改 `trap` 返回路径时必须考虑系统调用框架随后还会对寄存器进行哪些操作。

实验结果

完成三个任务后，在 `traps` 分支运行：

```
$ make grade
```

测试内容包括 `answers-traps.txt`、`backtrace`、`alarmtest` 的四个子测试、`usertests-q` 和运行时间检查。最终所有测试均通过，结果如下图所示。

```
jambitx@Johnny:~/Course/xv6-labs-2025$ make grade
make[1]: Leaving directory '/home/jambitx/Course/xv6-labs-2025'
== Test answers-traps.txt ==
answers-traps.txt: OK
== Test backtrace test ==
$ make qemu-gdb
backtrace test: OK (8.2s)
== Test running alarmtest ==
$ make qemu-gdb
(5.6s)
== Test alarmtest: test0 ==
alarmtest: test0: OK
== Test alarmtest: test1 ==
alarmtest: test1: OK
== Test alarmtest: test2 ==
alarmtest: test2: OK
== Test alarmtest: test3 ==
alarmtest: test3: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (94.8s)
== Test time ==
time: OK
Score: 95/95
```

图 8 Traps Lab 的 make grade 测试结果

本实验依次从指令、内核栈和 `trapframe` 三个角度分析程序执行状态。RISC-V assembly 任务说明寄存器和跳转指令如何完成函数调用；Backtrace 任务根据栈帧中保存的信息恢复内核调用链；Alarm 任务在时钟中断到来时保存用户现场、改变返回地址，并通过 `sigreturn()` 恢复原状态。最终测试通过表明，新增调试功能和定时告警机制没有破坏 xv6 原有的系统调用、进程运行和用户态返回流程。

Lab5: Copy-on-Write Fork

对应源码分支：<https://github.com/JambitX11/xv6-labs-2025/tree/cow>

本实验实现 xv6 的 copy-on-write fork。传统操作系统课程中通常会先介绍“进程拥有独立地址空间”“页表完成虚拟地址到物理地址的转换”“fork 会复制父进程”等概念，但真正进入 xv6 代码后，容易出现的困难是：题目只说要实现 COW fork，而代码分散在进程管理、页表管理、物理内存分配和异常处理多个文件中，新手很难判断从哪里开始。

本实验可以先从题目语义反推代码路径。`fork()` 的外部语义是创建一个几乎和父进程相同的子进程；用户程序应该看到父子进程初始内存内容相同，之后各自写入互不影响。原始 xv6 为了满足这个语义，在 fork 时立即复制所有用户物理页。COW 并不改变这个语义，只改变实现时机：fork 时先让父子进程共享同一批物理页，并把这些页临时设为只读；当其中一方真正写入某一页时，处理器触发 page fault，内核再为写入者复制这一页。

因此，定位代码时可以沿着下面的执行链查找：

```
用户程序调用 fork()
-> kernel/proc.c 中的 kfork()
-> kernel/vm.c 中的 uvmcopy()
-> 原本在这里复制用户物理页

用户程序写入 COW 页
-> PTE_W 被清除，处理器触发 store page fault
-> kernel/trap.c 中的 usertrap()
-> 内核检查该页是否是 COW 页，并完成复制

进程退出或释放用户内存
-> kernel/vm.c 中的 uvmunmap()/uvmfree()
-> kernel/kalloc.c 中的 kfree()
-> 引用计数为 0 时才真正释放物理页
```

从这个角度看，本实验并不是在某一个函数中增加一段优化，而是在 xv6 原有内存管理路径上补全“共享页”“写时复制”和“共享页释放”三件事。

Copy-on-Write fork (hard)

实验目的

本任务要求将 xv6 中的 `fork()` 改造成 copy-on-write fork。原始 `fork()` 在创建子进程时会完整复制父进程的用户内存，这种实现简单但开销较大。尤其在 shell 启动程序的常见流程中，子进程往往在 fork 后立即执行 `exec()`，原先复制的用户内存很快被新程序替换，造成大量不必要的物理页分配和内存复制。

COW fork 的目标是在保持用户可见行为不变的前提下推迟复制。fork 时父子进程先共享物理页，页表项均被设置为只读并标记为 COW；当父进程或子进程尝试写入共享页时，内核再复制该物理页，并把当前进程的页表项改为指向新的可写页。这样，如果某些页面从未被写入，就完全不需要复制。

该任务同时要求正确维护物理页生命周期。COW 后同一个物理页可能被多个进程页表引用，释放页面时不能只根据当前进程判断是否回收物理页，而必须使用引用计数确认是否已经没有任何页表指向它。

实验步骤

实验首先需要明确 xv6 中与 COW 有关的几类对象。`struct proc` 表示进程，里面的 `pagetable` 指向该进程的用户页表；页表项 PTE 记录虚拟页映射到哪个物理页，以及该页是否可读、可写、可执行、是否允许用户态访问；`kalloc()` 和 `kfree()` 管理实际的 4KB 物理页；`usertrap()` 是用户态异常进入内核后的处理入口。

根据这些对象的分工，实验实现分为五个主要步骤。

第一，在 `kernel/riscv.h` 中增加 `PTE_COW` 标志位。RISC-V 的 PTE 除了硬件定义的 `PTE_V`、`PTE_R`、`PTE_W`、`PTE_X`、`PTE_U` 以外，还有保留给软件使用的位。实验使用其中一位标记“这是一个 COW 页”。这个标记是必要的，因为普通只读页和 COW 只读页不能混淆。代码段本来就是只读的，写它应当视为非法；COW 页只是临时清除了写权限，写入时应该复制后继续执行。

第二，在 `kernel/kalloc.c` 中为物理页增加引用计数。原始 xv6 的物理页分配器只维护空闲链表，`kalloc()` 从链表取出一页，`kfree()` 把一页放回链表。COW 引入共享后，单个物理页可能同时被父进程和一个或多个子进程映射，因此需要用数组记录每个物理页当前被多少个页表引用。

实现时，`kalloc()` 分配新页后将该页引用计数设为 1；`uvmcopy()` 让子进程共享父进程物理页时调用新增的引用计数增加函数；`kfree()` 不再直接释放页面，而是先将引用计数减 1，只有当计数降为 0 时，才把页面填充为调试用字节并放回空闲链表。由于多个进程可能并发 `fork`、`exit` 或释放内存，引用计数的读写需要受 `kmem.lock` 保护。

第三，修改 `kernel/vm.c` 中的 `uvmcopy()`。这是 `fork` 复制用户地址空间的核心函数，也是 COW 改造的主要入口。原始实现会遍历父进程地址空间，对每个已映射页面执行 `kalloc()`、`memmove()` 和 `mappages()`，也就是为子进程分配新物理页并复制内容。COW 实现去掉了这个立即复制过程，改为让子进程的 PTE 直接指向父进程原来的物理页。

处理每个 PTE 时，若该页原来带有 `PTE_W`，说明它是用户程序语义上可以写入的页。实验将父进程和子进程对应 PTE 都清除 `PTE_W`，同时加上 `PTE_COW`。这样父子进程继续读取该页时不会有额外开销，一旦写入，处理器会因为缺少写权限触发 `page fault`。若该页原本不可写，例如代码页，则保持只读共享即可，不需要设置 `PTE_COW`。每成功建立一个共享映射，都增加对应物理页的引用计数。修改父进程 PTE 后还需要刷新 TLB，避免处理器继续使用旧的可写权限缓存。

第四，修改 `kernel/trap.c` 中的 `usertrap()`，处理写 COW 页导致的 `store page fault`。RISC-V 中 `scause` 为 15 表示 `store page fault`，`stval` 保存发生异常的虚拟地址。实验在原有 lazy allocation 的 `vmfault()` 判断之前加入 COW 判断：如果这次异常是用户写 COW 页，就调用 COW 处理函数；如果不是 COW，再交给原有懒分配逻辑或按非法访问处理。

COW 处理函数位于 `kernel/vm.c`。它先将 `fault address` 按页对齐，再通过 `walk()` 找到 PTE，并检查该 PTE 是否同时满足有效、用户页和 COW 标记。若检查失败，说明这不是合法 COW 写入。若检查成功，继续查看原物理页引用计数。引用计数大于 1 时，当前进程必须分配一页新的物理内存，将旧页内容复制过去，随后把当前进程的 PTE 改为指向新页，并恢复 `PTE_W`、清除 `PTE_COW`。旧物理页的引用计数随之减少。若引用计数已经是 1，则说明当前进程是唯一引用者，不需要真正复制，只需恢复写权限并清除 COW 标记即可。

该流程可以概括为：

写入 COW 页

- > `usertrap()` 识别 `store page fault`
- > 找到 `fault address` 对应的 PTE
- > 检查 `PTE_COW`
- > 若共享者超过一个，分配新页并复制内容
- > 当前进程 PTE 指向新页，恢复 `PTE_W`
- > 返回用户态，重新执行原来的写指令

第五，修改 `kernel/vm.c` 中的 `copyout()`。这是本实验容易遗漏的一点。用户程序自己写 COW 页时会通过 `page fault` 进入 `usertrap()`；但 `copyout()` 是内核代表用户进程向用户地址写数据，例如 `read()` 把文件内容写入用户缓冲区，或者 `wait()` 把子进程退出状态写入用户地址。这个写入发生在内核代码中，不能只依赖用户态 `store page fault` 路径。

因此，`copyout()` 在真正执行 `memmove()` 前需要检查目标用户页是否带有 `PTE_COW`。如果是，就主动调用同一套 COW 处理函数，把目标页转成当前进程自己的可写页，再执行复制。若处理后仍没有写权限，则说明目标地址实际是普通只读页，应返回错误。这样用户态直接写入和内核替用户写入都遵守同一条规则：只要写 COW 页，先复制再写。

本实验主要修改文件及其作用如下：

- `kernel/riscv.h`：新增 `PTE_COW` 软件标志位，用于区分 COW 只读页和普通只读页。
- `kernel/kalloc.c`：为物理页增加引用计数，修改 `kalloc()`、`kfree()`，并提供增加、查询引用计数的辅助函数。
- `kernel/defs.h`：声明新增的引用计数函数和 COW 页处理函数，供不同内核文件调用。
- `kernel/vm.c`：修改 `uvmcopy()`，实现 `fork` 时共享物理页；新增 COW `fault` 处理函数；修改 `copyout()`，处理内核向用户 COW 页写入的情况。
- `kernel/trap.c`：在 `usertrap()` 中识别 `store page fault`，并优先处理 COW 页写入。

实验中遇到的问题和解决方法

本实验的第一个难点是判断“题目要求究竟落在哪些文件上”。如果只从 `fork()` 出发，容易认为只需要修改 `kernel/proc.c`。实际阅读代码后可以发现，`kfork()` 只是组织进程创建流程，真正复制用户地址空间的工作委托给 `uvmcopy()`。因此核心修改点不是 `kfork()` 本身，而是 `kernel/vm.c` 中的用户页表复制逻辑。

第二个难点是区分 COW 页和普通只读页。最初只看到写权限被清除时，容易把所有不可写页面都当成 COW 处理。这样会错误允许用户写代码段或其他本来只读的页面。解决方法是在 `fork` 时只对原来带 `PTE_W` 的用户页设置 `PTE_COW`，`page fault` 时也必须检查该标志。没有 `PTE_COW` 的只读页仍然按非法写入处理。

第三个难点是物理页释放。COW 后父子进程的页表可能指向同一个物理页，某个进程退出时只能解除自己的映射，不能直接释放物理页。为此在 `kalloc.c` 中加入引用计

数，并让 `kfree()` 自己决定是否真正回收页面。这样 `uvmunmap()`、`uvmfree()` 等原有路径仍然可以调用 `kfree()`，但释放动作会被引用计数保护起来。

第四个问题是 `copyout()`。仅在 `usertrap()` 中处理 COW fault 后，用户态普通写入可以工作，但涉及管道、文件读取、`wait()` 等系统调用时，内核可能需要向用户缓冲区写数据。测试程序 `cowtest` 中的 `filetest()` 正是检查这一点。解决方法是在 `copyout()` 中复用 COW 处理逻辑，使内核写用户地址前也能完成写时复制。

最后还需要注意 TLB 缓存。页表项权限被修改后，处理器可能仍缓存旧的地址翻译结果。实验在修改 COW 页权限后调用 `sfence_vma()` 刷新 TLB，保证后续访问按照新的 PTE 权限执行。

实验心得

COW fork 展示了操作系统中一个重要思想：用户可见的语义和内核内部的实现方式可以分离。用户程序看到的仍然是 fork 后父子进程拥有独立内存；内核内部却可以在安全的前提下临时共享物理页，并利用页表权限和 page fault 把复制动作延迟到真正写入时。

从代码阅读角度看，本实验也说明 xv6 虽然规模比课堂伪代码大，但可以按“语义入口、数据结构、异常路径、资源释放”逐步定位。`fork()` 告诉我们语义入口在哪里，`uvmcopy()` 告诉我们地址空间如何复制，PTE 权限位告诉我们如何制造写入异常，`usertrap()` 告诉我们异常如何回到内核，`kalloc.c` 则决定物理页何时真正释放。把这些路径连起来后，实验目标就不再是零散地修改若干文件，而是实现一条完整的内存管理闭环。

本实验还加深了对 page fault 的理解。缺页异常并不一定表示程序出错，它也可以是内核有意设计的控制点。COW 正是通过“先撤销写权限，再在写入异常中补做复制”来减少 fork 的开销。引用计数和 `copyout()` 的处理则提醒我们，内核机制的正确性不仅取决于正常路径，还取决于退出、释放、系统调用写回等边界路径是否保持一致。

实验结果

完成本 Lab 后，在 `cow` 分支运行：

```
$ make grade
```

最终 `cowtest`、`usertests -q` 和相关评分项均通过，得分为满分。测试结果如下图所示。

```
jambitx@Johnny:~/Course/xv6-labs-2025$ make grade
make[1]: Leaving directory '/home/jambitx/Course/xv6-labs-2025'
== Test running cowtest ==
$ make qemu-gdb
(32.9s)
== Test simple ==
simple: OK
== Test three ==
three: OK
== Test file ==
file: OK
== Test forkfork ==
forkfork: OK
== Test usertests ==
$ make qemu-gdb
(67.2s)
== Test usertests: copyin ==
usertests: copyin: OK
== Test usertests: copyout ==
usertests: copyout: OK
== Test usertests: all tests ==
usertests: all tests: OK
== Test time ==
time: OK
Score: 130/130
```

图 9 Copy-on-Write Lab 的 make grade 测试结果

测试通过表明，fork 时共享物理页、写 COW 页时复制、进程释放时按引用计数回收、以及 copyout() 写用户 COW 页等关键路径均符合实验要求。

Lab6: Networking

对应源码分支：<https://github.com/JambitX11/xv6-labs-2025/tree/net>

本实验实现 xv6 的网络通信功能。相比前面的系统调用、页表、trap 和 COW 实验，Networking Lab 面对的是另一类内核问题：内核如何与外部设备协作。课堂上的操作系统概念通常会讲“设备驱动”“中断”“DMA”“网络协议栈”，但进入 xv6 代码后，真正困难的是判断这些概念分别落在哪个文件、哪条调用路径和哪组数据结构上。

题目本身分为两个任务点。第一部分是 NIC，即 Network Interface Card，要求完成 E1000 网卡驱动的发送和接收函数。此时 xv6 只需要把完整 Ethernet frame 交给网卡，或从网卡取回完整 frame；驱动不需要理解 UDP 载荷内容。第二部分是 UDP Receive，要求在 kernel/net.c 中接收 UDP 包、按目的端口排队，并允许用户进程通过 recv() 读取。二者合起来构成一条完整路径：

```
宿主机或用户程序产生 UDP 数据
-> Ethernet/IP/UDP packet
-> E1000 描述符环
```

```
-> xv6 网络栈
-> 按 UDP 目的端口入队
-> 用户进程 recv() 取走 payload
```

因此，本实验不是从零实现完整 TCP/IP 协议栈，而是在 xv6 已提供的简化网络协议代码和 QEMU 模拟网卡之间补上关键连接。做题时应先区分“设备驱动层”和“协议/用户接口层”，再分别沿发送和接收数据流定位代码。

Part One: NIC (moderate)

实验目的

本任务要求完成 kernel/e1000.c 中的 e1000_transmit() 和 e1000_recv(), 使 xv6 能够通过 QEMU 模拟的 Intel E1000 网卡发送和接收 packet。QEMU 为 xv6 提供了一个虚拟局域网环境，xv6 的 IP 地址为 10.0.2.15，宿主机地址为 10.0.2.2。在这一部分中，上层网络代码已经能够构造或处理 Ethernet frame，实验需要补齐的是 frame 与 E1000 设备之间的交接。

E1000 驱动的核心不是逐字节读写网络数据，而是操作描述符环。描述符环是一组环形数组项，每个 descriptor 描述一个 packet buffer 的地址、长度和状态。发送方向上，驱动把待发送 buffer 写入发送描述符，并移动 E1000_TDT 通知硬件；接收方向上，硬件通过 DMA 把数据写入接收描述符指向的 buffer，再设置完成位并触发中断。

从题目到代码定位，可以沿着以下路径理解：

```
发送 packet
-> kernel/net.c 构造 Ethernet frame
-> e1000_transmit(buf, len)
-> tx_ring 和 E1000_TDT

接收 packet
-> E1000 触发中断
-> e1000_intr()
-> e1000_recv()
-> rx_ring 和 net_rx(buf, len)
```

实验步骤

首先阅读 kernel/e1000.c 中已有的 e1000_init()。初始化代码已经完成设备复位、发送环和接收环的配置、MAC 地址过滤、中断开启等工作，并设置了两个关键数组：tx_ring 和 rx_ring。这说明本任务不需要重新设计设备初始化流程，而是要在已有 ring 的基础上补齐运行时收发逻辑。

发送路径中，为每个发送 descriptor 增加 tx_bufs 记录数组，用来保存该 descriptor 当前挂着的内核 buffer。e1000_transmit() 先读取 E1000_TDT，找到当前软件准备使用的发送槽位，然后检查该 descriptor 的 E1000_TXD_STAT_DD 位。如果 DD 位没有设置，说明网卡尚未完成该槽位上一次发送，函数返回 -1，由调用者释放本次 buffer。若 DD 位已经设置，并且 tx_bufs 中保存了旧 buffer，则说明旧 packet 已经被网卡读取并发送完成，可以安全释放。

随后，发送函数把新 buffer 的地址写入 descriptor 的 addr 字段，把 frame 长度写入 length 字段，并设置 E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS。EOP 表示这是一个完整 packet 的结尾，RS 要求设备发送完成后回写状态位。写入新 descriptor 前需要清空旧 status，最后将 E1000_TDT 向后移动一格，通知硬件有新的发送任务。

这里不能在 e1000_transmit() 返回成功后立即释放新 buffer。因为 E1000 通过 DMA 异步读取内存，驱动更新 tail 之后，设备才会去读 descriptor 指向的内存。如果过早释放，物理页可能被分配给其他用途，网卡随后读到的内容就不再是原来的 packet。本实验通过 tx_bufs 把 buffer 生命周期延长到设备报告 descriptor done 之后。

接收路径中，e1000_recv() 从 (E1000_RDT + 1) % RX_RING_SIZE 开始检查。RDT 表示软件已经交还给硬件的最后一个 descriptor，因此下一项才可能是新收到的 packet。只要当前 descriptor 的 E1000_RXD_STAT_DD 位被设置，就说明网卡已经把一个 packet 写入该 descriptor 指向的 buffer。驱动取出该 buffer 和长度，立即为同一 descriptor 分配新的空页作为后续接收缓冲区，清除 status，更新 E1000_RDT，再把旧 buffer 交给 net_rx(buf, len)。

接收函数特别注意不在持有 e1000_lock 时调用 net_rx()。net_rx() 如果收到 ARP 请求，可能调用 arp_rx() 构造 ARP reply，而 arp_rx() 会再次进入 e1000_transmit()。若接收路径持锁调用上层网络代码，就可能在同一个中断处理过程中再次申请发送锁，导致死锁。因此本实验仅在发送路径保护发送 descriptor ring，接收路径在交给上层前完成 descriptor 更新。

本部分实际修改 kernel/e1000.c。新增的 tx_bufs 数组用于记录发送 buffer；e1000_transmit() 实现发送 descriptor 填写、发送完成检查、旧 buffer 释放和 tail 更新；e1000_recv() 实现接收 descriptor 扫描、buffer 替换、status 清理、tail 更新以及向 net_rx() 交付 packet。

实验中遇到的问题和解决方法

本部分的第一个难点是理解 descriptor 的状态位含义。DD 不是普通的软件标志，而是设备和驱动之间的同步协议。发送时，如果没有检查 E1000_TXD_STAT_DD 就覆盖 descriptor，可能破坏尚未完成的发送；接收时，如果没有检查 E1000_RXD_STAT_DD 就取 buffer，可能把空 buffer 当成有效 packet 交给上层。

第二个问题是发送 buffer 的释放时机。kernel/net.c 中 sys_send() 分配的 buffer 被写入发送 descriptor 后，网卡并不会同步完成发送，因此不能立即释放。解决

方法是在 `tx_bufs` 中保存每个 descriptor 的旧 buffer，并在下次复用同一 descriptor 且看到 DD 位时释放，从而保证设备 DMA 读取期间 buffer 仍然有效。

第三个问题是接收路径的锁使用。若在 `e1000_recv()` 中持有 `e1000_lock` 调用 `net_rx()`，收到 ARP 请求时 `arp_rx()` 可能立即调用 `e1000_transmit()` 发送回复，从而再次申请 `e1000_lock`。解决方法是让接收函数只负责更新接收 descriptor，并在不持发送锁的情况下调用上层网络处理函数。

实验心得

NIC 部分展示了设备驱动开发的基本特点：代码本身并不长，但必须准确遵守硬件约定。descriptor 的地址字段、长度字段、DD 位、EOP 和 RS 命令位、tail 寄存器都不是普通变量，而是内核和设备之间共享的协议。任何一个状态更新顺序错误，都可能表现为丢包、重复使用 buffer、死锁或内存泄漏。

本部分也说明，驱动层并不负责理解 UDP 或 DNS 的语义。它只关心完整 Ethernet frame 在哪块内存、长度是多少、设备是否已经发送或接收完成。把驱动层职责限定清楚后，`kernel/e1000.c` 的修改就集中在描述符环和 buffer 生命周期上。

Part Two: UDP Receive (moderate)

实验目的

本任务要求在 `kernel/net.c` 中实现 UDP 接收。UDP 允许不同主机上的进程通过 IP 地址和端口号交换单个 datagram。一个 UDP packet 包含源端口和目的端口；接收端内核需要根据目的端口判断该 packet 应交给哪个用户进程。题目已经提供了构造并发送 UDP packet 的大部分代码，第二部分要补齐的是：收到 IP packet 后解析 UDP 头，将 payload 按目的端口排队，并让用户进程通过 `recv()` 读取。

这一部分的核心不是 E1000 硬件寄存器，而是内核中的协议分发和进程等待机制。用户调用 `bind(port)` 表示愿意接收该端口上的 UDP packet；如果 packet 到达时已经有进程在 `recv()` 中等待，内核应唤醒它；如果 packet 先到达而用户稍后调用 `recv()`，内核应暂时把 packet 放入有限队列。

实验步骤

首先在 `kernel/net.c` 中新增 UDP 端口队列结构。每个队列对应一个已经绑定的目的端口，队列节点保存一个已经收到但尚未被用户取走的 UDP packet，包括 payload buffer、payload 长度、源 IP、源端口和 next 指针。队列数量和每个端口的队列长度都设置为有限值，避免某个端口长时间不读取时无限消耗物理页。

`sys_bind()` 读取用户传入的端口号，在全局 UDP 队列表中查找是否已有绑定；若没有，则分配一个空队列槽位并初始化。重复绑定同一端口时直接返回成功。`sys_unbind()`

释放对应端口队列中所有尚未读取的 packet，并清空队列状态。由于网络中断路径和用户系统调用路径都可能访问这些队列，所有队列操作都使用 `netlock` 保护。

随后实现 `ip_rx()`。`net_rx()` 已经根据 Ethernet 类型区分 ARP 和 IP，ARP packet 由已有 `arp_rx()` 处理，IP packet 进入 `ip_rx()`。实验在 `ip_rx()` 中检查 IP 版本、IP 头长度、总长度和协议号，只处理 UDP packet。解析 UDP 头后，取得目的端口、源端口和 payload 长度，将 payload 移到 buffer 开头，并构造队列节点保存源地址、源端口和 payload 信息。若目的端口没有绑定，或对应队列已经达到上限，则释放该 packet；否则追加到队尾，并调用 `wakeup(q)` 唤醒睡在该端口队列上的 `recv()` 进程。

接着实现 `sys_recv()`。该系统调用读取目的端口、源 IP 输出地址、源端口输出地址、用户缓冲区地址和最大长度。若端口没有绑定，则返回错误；若队列为空，则通过 `sleep(q, &netlock)` 阻塞等待。`sleep()` 会在进入睡眠时释放 `netlock`，被 `ip_rx()` 唤醒后再重新获得锁，因此不会阻塞网络中断继续入队。取到 packet 后，`sys_recv()` 从队列头删除该节点，释放锁，然后使用 `copyout()` 将源 IP、源端口和最多 `maxlen` 字节 payload 写入用户地址空间，最后释放内核中的 packet 节点和 payload buffer。

本部分还修改 `sys_send()` 的错误处理。`sys_send()` 已经能够构造 Ethernet、IP 和 UDP 头部，并把用户缓冲区中的 payload 复制到内核 buffer 中。实验将其末尾改为检查 `e1000_transmit()` 的返回值：若发送 ring 暂无可用 descriptor，则释放刚分配的 buffer 并返回 -1。这虽然和接收队列不属于同一方向，但属于网络 buffer 生命周期管理的一部分，可以避免发送失败时泄漏物理页。

UDP 接收流程可以概括为：

```
E1000 收到 Ethernet frame
-> e1000_recv() 交给 net_rx()
-> net_rx() 判断 Ethernet 类型
-> ip_rx() 解析 IP/UDP 头
-> 根据 UDP 目的端口找到接收队列
-> 入队并 wakeup 等待的 recv()
-> sys_recv() copyout 源地址、源端口和 payload
```

本部分实际修改 `kernel/net.c`。新增 UDP 队列和 packet 节点结构；实现 `sys_bind()`、`sys_unbind()`、`sys_recv()` 和 `ip_rx()`；并补充 `sys_send()` 的发送失败清理。实验过程中还重点阅读了 `kernel/net.h` 中的 Ethernet、IP、UDP、ARP 和 DNS 结构定义，以及 `user/nettest.c`、`nettest.py` 中对收包、发包、多端口分发、DNS 和内存释放的测试逻辑。

实验中遇到的问题和解决方法

本部分的第一个难点是区分协议层和驱动层职责。刚开始容易以为 UDP Receive 需要修改 E1000 接收逻辑。实际数据流显示，`e1000_recv()` 只交付完整 Ethernet frame，

UDP 目的端口分发应在 `kernel/net.c` 的 `ip_rx()` 中完成。明确层次后，`kernel/net.c` 中的逻辑就集中在协议头解析、端口队列和用户进程唤醒上。

第二个问题是 UDP 队列不能无限增长。`ping3` 测试会从部分端口持续发送大量 packet，并检查是否出现过多排队。若每个收到的 UDP 包都无限入队，短期内看似不丢包，但会消耗大量物理页，并导致最终 `free` 测试失败。实验为每个端口设置固定队列上限，超过上限时主动丢弃并释放 packet。

第三个问题是阻塞式 `recv()` 的退出条件。若用户进程在没有 packet 时睡眠等待，而另一个进程对其执行 `kill()`，`recv()` 必须能检测 `killed(p)` 并返回错误，否则测试中用于统计队列长度的子进程无法退出。实验在等待循环中检查进程 `killed` 状态，从而使阻塞系统调用可以被终止。

第四个问题是内核和用户空间的数据复制。`recv()` 返回的不应是整个 Ethernet frame，而是 UDP payload，同时还要把源 IP 和源端口写回用户提供的地址。实验在 `ip_rx()` 中将 payload 移到 buffer 开头，在 `sys_recv()` 中使用 `copyout()` 分别写回源地址、源端口和 payload，从而保持用户接口与 `user/nettest.c` 的期望一致。

实验心得

UDP Receive 部分体现了网络协议栈中“分发”的思想。网卡只提供一串收到的字节，内核需要逐层解释 Ethernet、IP 和 UDP 头部，最终根据 UDP 目的端口把 payload 放入对应队列。用户进程看到的是 `bind()` 和 `recv()` 这样的接口，但背后需要中断处理、协议解析、队列保护和阻塞唤醒共同完成。

从代码定位角度看，本部分说明面对一个较大的内核项目时，应先按数据流划分边界。接收路径从设备中断开始，经过驱动取包、协议解析、端口排队，最终由用户 `recv()` 取走。沿着这条路径阅读代码，问题就从“网络代码很多不知道看哪里”变成了“每一段代码负责把 packet 交给下一层”。

本部分也再次体现了内存生命周期的重要性。收到但无人接收的 packet 需要暂存，队列满或端口未绑定时必须释放，`unbind()` 时也要释放已排队数据。否则功能测试可能暂时通过，但最终会在 `free` 测试中暴露内存泄漏。

实验结果

完成本 Lab 后，在 `net` 分支运行：

```
$ make grade
```

测试过程中，评分脚本在宿主机启动 `nettest.py grade`，同时在 `xv6` 内运行 `nettest grade`。最终 `txone`、`arp_rx`、`ip_rx`、`ping0`、`ping1`、`ping2`、`ping3`、`dns`、`free` 和 `time` 测试均通过，得分为满分。测试结果如下图所示。

```
jambitx@Johnny: ~/Course/xv6 × + ▾
make[1]: Leaving directory '/home/jambitx/Course/xv6-labs-2025'
== Test running nettest ==
$ make qemu-gdb
(21.6s)
== Test nettest: txone ==
nettest: txone: OK
== Test nettest: arp_rx ==
nettest: arp_rx: OK
== Test nettest: ip_rx ==
nettest: ip_rx: OK
== Test nettest: ping0 ==
nettest: ping0: OK
== Test nettest: ping1 ==
nettest: ping1: OK
== Test nettest: ping2 ==
nettest: ping2: OK
== Test nettest: ping3 ==
nettest: ping3: OK
== Test nettest: dns ==
nettest: dns: OK
== Test nettest: free ==
nettest: free: OK
== Test time ==
time: OK
Score: 171/171
jambitx@Johnny: ~/Course/xv6-labs-2025$ |
```

图 10 Networking Lab 的 make grade 测试结果

测试通过表明，E1000 发送描述符环、接收描述符环、ARP/IP/UDP 分发、UDP 端口队列、阻塞接收和网络 buffer 释放等关键路径均符合实验要求。

Lab7: Lock

对应源码分支：<https://github.com/JambitX11/xv6-labs-2025/tree/lock>

本实验围绕 xv6 内核中的锁展开。前面的实验更多关注“怎样实现一个功能”，例如系统调用、页表映射、异常处理或网络收发；Lock Lab 则进一步关注“功能已经正确时，内核在多核环境下是否还能高效、可扩展地运行”。在多核操作系统中，锁是保护共享数据结构的基本工具，但锁本身也可能成为性能瓶颈。如果所有 CPU 都频繁等待同一把锁，那么即使机器有多个 CPU，实际运行效果也会接近串行执行。

本实验包含两个任务点。第一个任务是优化物理内存分配器，将原来所有 CPU 共用的一条空闲页链表拆分为每 CPU 一条空闲链表，从而减少 `kalloc()` 和 `kfree()` 对同一把 `kmem` 锁的竞争。第二个任务是实现读写自旋锁，使多个读者可以并发进入临界区，而写者仍然保持独占，并且在有写者等待时阻止新的读者插队，避免写者长期饥饿。

这两个任务体现的是同一类内核设计问题：锁不仅要保证正确性，还要尽量缩小不必要的共享范围。内存分配器任务通过拆分数据结构减少锁竞争，读写锁任务通过区分读路径和写路径提高并发度。

Memory allocator (moderate)

实验目的

本任务要求优化 xv6 的物理内存分配器，降低多 CPU 同时执行 `kalloc()` 和 `kfree()` 时对 `kmem` 锁的竞争。原始 xv6 在 `kernel/kalloc.c` 中维护一个全局空闲页链表，所有物理页分配和释放都必须经过同一个 `kmem.lock`。这种设计实现简单，但在多核环境中会造成明显瓶颈：多个 CPU 即使操作的是不同物理页，也必须轮流抢同一把锁。

`kalloctest` 正是用来暴露这个问题的测试程序。它通过多个进程反复增长和收缩地址空间，制造大量 `sbrk()`、`kalloc()` 和 `kfree()` 调用，并统计每把锁在 `acquire()` 中执行 `test-and-set` 但失败的次数。这个次数越高，说明锁竞争越激烈。实验目标是在不破坏内存分配正确性的前提下，让 `kmem` 相关锁的争用次数显著下降。

从概念上看，本任务不是删除锁，而是把“一把保护所有空闲页的大锁”改造成“每个 CPU 各自维护一条空闲页链表和一把锁”。大多数情况下，CPU 只访问自己的 `freelist`；只有当前 CPU 没有空闲页时，才从其他 CPU 的 `freelist` 中偷取空闲页。

实验步骤

实验主要修改 `kernel/kalloc.c`。原始代码中 `kmem` 是单个全局结构体，包含一把锁和一条 `freelist`。实验将其改为以 `NCPU` 为长度的数组：

```
struct {  
    struct spinlock lock;  
    struct run *freelist;  
} kmem[NCPU];
```

这样每个 CPU 都有独立的 `kmem[id].lock` 和 `kmem[id].freelist`。`NCPU` 定义在 `kernel/param.h` 中，表示 xv6 支持的最大 CPU 数量。`kinit()` 中需要循环初始化每一把锁，并保证锁名以 "kmem" 开头，因为 `statistics()` 系统调用会按照锁名统计 `kmem` 相关锁的争用情况：

```
for(int i = 0; i < NCPU; i++)  
    initlock(&kmem[i].lock, "kmem");
```

释放页面时，`kfree()` 不再把页放回全局 `freelist`，而是放回当前 CPU 对应的 `freelist`。由于 `cpuid()` 只有在中断关闭时才能安全调用，因此 `kfree()` 需要先使用 `push_off()` 关闭中断，取得 CPU 编号后再操作对应链表：

```
push_off();  
int id = cpuid();  
acquire(&kmem[id].lock);  
r->next = kmem[id].freelist;  
kmem[id].freelist = r;  
release(&kmem[id].lock);  
pop_off();
```

分配页面时，`kalloc()` 也首先访问当前 CPU 的 `freelist`。如果本地 `freelist` 非空，就直接取出头结点并返回，这条路径只需要竞争本 CPU 的锁。当本地 `freelist` 为空时，再调用偷页逻辑从其他 CPU 获取空闲页。

本实验最终采用的偷页策略是：当前 CPU 缺页时，从其他 CPU 的 `freelist` 中找一条非空链表，将其整体摘下；第一张页返回给当前这次 `kalloc()`，剩余页面挂到当前 CPU 的 `freelist`。这个策略的关键是缩短持有其他 CPU 锁的时间。偷页时只需要执行：

```
r = kmem[i].freelist;
if(r)
    kmem[i].freelist = 0;
```

也就是拿到锁、摘下链表头、清空对方 `freelist`、释放锁。它不在持锁期间遍历长链表，因此可以显著降低 `test-and-set` 失败次数。随后再把摘下链表中的第一张页与剩余部分拆开，第一张页给本次分配，剩余部分作为当前 CPU 后续分配的本地缓存。

整体执行逻辑可以概括为：

```
kfree(pa)
-> 关闭中断并读取当前 CPU 编号
-> 将 pa 插入当前 CPU 的 freelist

kalloc()
-> 关闭中断并读取当前 CPU 编号
-> 优先从当前 CPU 的 freelist 取页
-> 若本地为空，则从其他 CPU 的 freelist 偷取
-> 返回一个 4KB 物理页
```

本任务实际修改文件为 `kernel/kalloc.c`。其中 `kmem` 数据结构被改为每 CPU 数组；`kinit()` 初始化每 CPU 锁；`kfree()` 将释放的物理页放回当前 CPU 的链表；`kalloc()` 本地优先分配，并在缺页时调用偷页函数；新增的偷页函数负责在本地 `freelist` 为空时从其他 CPU 转移空闲页。

实验中遇到的问题和解决方法

本任务调试的主要问题出现在 `kalloc_test` 的 `test4`。最初虽然已经把全局 `kmem` 拆分为每 CPU `freelist`，但 `test4` 仍然失败，输出中的 `tot` 明显高于测试允许的阈值。这说明问题已经不是内存分配正确性，而是锁争用仍然过高。

第一次尝试中，偷页函数采用固定批量偷页，例如一次偷 32、64、1024 页。小批量偷页会导致当前 CPU 在压力测试中频繁访问其他 CPU 的 `freelist`；大批量偷页则会在持有其他 CPU 的 `kmem` 锁时遍历较长链表。两种方式都会导致 `#test-and-set` 次数偏高，只是表现形式不同。尤其当批量过大时，临界区变长，其他 CPU 更容易在 `acquire()` 中自旋。

最终解决方法是将偷页逻辑改为“整体摘下对方 freelist”。这样持有受害 CPU 锁的时间极短，不需要在锁内遍历链表；同时当前 CPU 获得一批本地可用页，后续大量 `kalloc()` 可以在本 CPU freelist 内完成。修改后 `kalloctest` 的 `test1` 和 `test4` 均通过，说明该实现同时满足正确性和低争用要求。

本任务还需要注意 `cpuid()` 的使用位置。`cpuid()` 依赖当前 CPU 状态，若在中断开启时调用，当前进程可能因中断和调度而迁移到其他 CPU，导致得到的 CPU 编号不可靠。因此 `kalloc()` 和 `kfree()` 都在 `push_off()` 和 `pop_off()` 包围范围内读取并使用 CPU 编号。

实验心得

物理内存分配器任务说明，锁优化的本质不是简单地减少 `acquire()` 调用次数，而是减少多个 CPU 对同一共享状态的争用。原始 xv6 使用一个全局 freelist，所有 CPU 都必须围绕同一个数据结构同步；改成每 CPU freelist 后，大多数分配和释放都变成了本地操作，只有本地资源耗尽时才需要跨 CPU 协调。

本任务也展示了性能测试和正确性测试的区别。内存页没有丢失、`usertests` 能通过，只能说明分配器基本正确；`kalloctest` 中的 `test-and-set` 统计则进一步检查实现是否真正降低了锁竞争。调试过程中，固定批量偷页虽然在功能上可行，但在压力测试下仍会暴露临界区过长或跨 CPU 偷页过频的问题。最终通过缩短偷页时持锁时间，才达到实验要求。

Read-write lock (moderate)

实验目的

本任务要求在 xv6 中实现读写自旋锁。普通 `spinlock` 只提供互斥语义：只要一个 CPU 持有锁，其他 CPU 无论是读还是写都不能进入临界区。然而在很多内核场景中，共享数据以读取为主，多个读者之间并不会互相破坏状态。例如题目中提到的 `ticks` 变量，多个系统调用读取 `ticks` 时彼此不冲突，真正需要互斥的是读写并发和写写并发。

读写锁的语义可以概括为三条：多个 `reader` 可以同时持有锁；`writer` 必须独占锁；当 `writer` 已经在等待时，新的 `reader` 不能继续插队。最后一条用于避免 `writer starvation`。如果读者源源不断进入，写者可能永远等不到 `readers == 0` 的时刻，因此实验要求实现 `writer priority`。

实验步骤

本任务主要修改 `kernel/spinlock.h` 和 `kernel/spinlock.c`。`kernel/spinlock.c` 已经提供了读写锁 API 的外层函数：

```
void read_acquire(struct rwspinlock *rwlk);
void read_release(struct rwspinlock *rwlk);
void write_acquire(struct rwspinlock *rwlk);
void write_release(struct rwspinlock *rwlk);
void initrwlock(struct rwspinlock *rwlk);
```

外层 `read_acquire()`、`write_acquire()` 会调用 `push_off()` 关闭中断,外层 `release` 函数会调用 `pop_off()` 恢复中断。因此实验只需要实现 `inner` 函数和读写锁内部状态,不需要在 `inner` 函数中再次关闭或恢复中断。

在 `kernel/spinlock.h` 中,读写锁结构被改为:

```
struct rwspinlock {
    uint state;
    uint waiting_writers;
};
```

其中 `state` 同时编码 `writer` 状态和 `reader` 数量。实验使用最高位表示是否有 `writer` 正在持有锁,低 31 位表示当前 `reader` 数量:

```
#define RW_WRITER 0x80000000U
#define RW_READERS 0x7fffffffU
```

这种设计的好处是, `reader` 增加计数和 `writer` 获得独占状态都围绕同一个原子变量 `state` 完成。 `reader` 只有在 `state` 中没有 `writer` 位时,才能通过 CAS 将 `state` 从 `s` 改为 `s + 1`; `writer` 只有在 `state == 0` 时,才能通过 CAS 将 `state` 改为 `RW_WRITER`。这样读写互斥和写写互斥都由同一个原子状态保证,避免多个独立变量之间出现竞态窗口。

`read_acquire_inner()` 的逻辑是:先等待没有 `writer` 正在持有锁,并且没有 `writer` 正在等待;随后读取 `state`,尝试通过 CAS 增加 `reader` 数量。CAS 成功后还需要再次检查 `waiting_writers`,因为可能出现 `reader` 刚刚增加计数、`writer` 同时到达并增加等待计数的情况。如果此时发现已有 `writer` 等待, `reader` 主动将 `state` 减回去并重新等待,从而保证 `writer priority`。

`write_acquire_inner()` 的逻辑是: `writer` 到达后先将 `waiting_writers` 加一,阻止新的 `reader` 进入;随后不断尝试将 `state` 从 0 原子地改为 `RW_WRITER`。只有当没有 `reader` 且没有其他 `writer` 时,这个 CAS 才能成功。成功后再将 `waiting_writers` 减一,表示该 `writer` 已经从等待状态变为持有写锁状态。

释放路径相对直接。`read_release_inner()` 通过 CAS 将 `reader` 数量减一,并检查不能在 `writer` 持有状态或 `reader` 数量为 0 时释放读锁。`write_release_inner()` 只允许将 `state` 从 `RW_WRITER` 改回 0,如果释放时状态不匹配,则说明调用顺序错误,直接 `panic`。

读写锁的整体状态转换可以概括为：

```
无持有者: state = 0
    reader acquire -> state = state + 1
    writer acquire -> state = RW_WRITER

多个 reader: state = reader 数量
    新 reader 在无等待 writer 时可以继续进入
    writer 到达后 waiting_writers 增加, 后续 reader 停止进入

writer 持有: state = RW_WRITER
    reader 和其他 writer 均等待
    writer release 后 state 回到 0
```

本任务实际修改文件为 kernel/spinlock.h 和 kernel/spinlock.c。spinlock.h 中重新定义 struct rwspinlock; spinlock.c 中实现 initrwlock()、read_acquire_inner()、read_release_inner()、write_acquire_inner() 和 write_release_inner(), 并通过 GCC 原子操作完成状态检查和更新。

实验中遇到的问题和解决方法

本任务调试过程中先后遇到两个典型问题。第一次实现采用内部普通 spinlock 保护 readers、writer 和 waiting_writers 三个字段。该版本虽然思路直观, 但在 rwlktest 中出现超时, 测试停在多个锁组合获取的阶段。这说明读写锁内部实现过于依赖普通 spinlock 或等待逻辑没有正确释放内部锁时, 容易在嵌套获取不同读写锁时造成长时间等待。

随后尝试使用三个独立原子变量分别表示 readers、writer 和 waiting_writers。这个版本不再超时, 但 rwlktest 只显示 3/4 CPUs succeeded。根据测试代码可以判断, 失败 CPU 很可能出现在写锁路径, 说明读者计数和写者状态分散在不同变量上时, 仍可能存在微小竞态窗口: reader 修改 readers 与 writer 修改 writer 并不是对同一个状态的一次原子转换, 测试在高并发循环中会捕捉到这种边界问题。

最终解决方法是将 reader 数量和 writer 状态合并到单个 state 字段。reader 和 writer 都通过 CAS 修改同一个变量, 因此不再依赖多个变量之间的组合判断来保证互斥。waiting_writers 只用于表达“已有 writer 排队”, 并通过 reader acquire 后的复查和回滚保证新 reader 不会插队到等待 writer 前面。修改后, rwlktest 中所有 4 个 CPU 均成功, 说明读读并发、读写互斥、写写互斥和 writer priority 均满足测试要求。

实验心得

读写锁任务说明, 并发控制中最困难的部分往往不是写出大致逻辑, 而是消除微小竞态窗口。用三个字段分别描述 reader、writer 和等待 writer, 在概念上很容易理解,

但如果这些字段不是作为一个整体被原子更新，就可能在极短时间内出现不一致状态。测试循环执行上百万次后，这类平时难以观察的问题就会暴露出来。

将状态合并为单个 `state` 后，读写锁的核心互斥关系变得更清晰：reader 获取锁就是将 `state` 加一，writer 获取锁就是将 `state` 从 0 改成 writer 标志。两者不能同时成功，因为它们竞争的是同一个原子变量。这种实现方式比多个变量配合判断更接近“把并发条件压缩为一次不可分割的状态转换”。

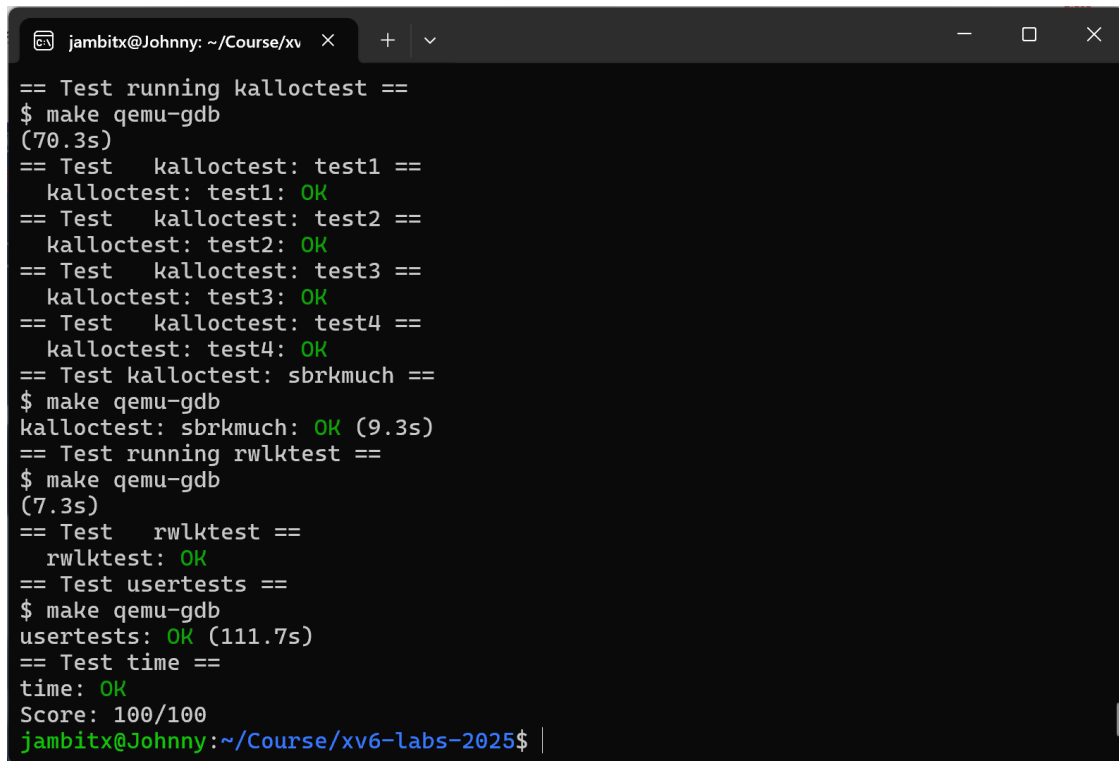
本任务也加深了对 writer priority 的理解。仅仅保证 writer 独占并不够，如果新 reader 可以不断进入，那么 writer 仍然可能长期无法执行。`waiting_writers` 的作用不是表示锁已经被 writer 持有，而是表达“从现在开始 reader 不应继续插队”。这种设计使已有 reader 能正常退出，同时阻止新的 reader 扩大读者集合，为等待中的 writer 创造进入临界区的机会。

实验结果

完成本 Lab 后，在 `lock` 分支运行：

```
$ make grade
```

最终 `kalloctest`、`rwlktest`、`usertests -q` 和 `time` 测试均通过，得分为满分。测试结果如下图所示。



```
jambitx@Johnny: ~/Course/xv  ×  +  v
== Test running kalloctest ==
$ make qemu-gdb
(70.3s)
== Test  kalloctest: test1 ==
kalloctest: test1: OK
== Test  kalloctest: test2 ==
kalloctest: test2: OK
== Test  kalloctest: test3 ==
kalloctest: test3: OK
== Test  kalloctest: test4 ==
kalloctest: test4: OK
== Test kalloctest: sbrkmuch ==
$ make qemu-gdb
kalloctest: sbrkmuch: OK (9.3s)
== Test running rwlktest ==
$ make qemu-gdb
(7.3s)
== Test  rwlktest ==
rwlktest: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (111.7s)
== Test time ==
time: OK
Score: 100/100
jambitx@Johnny:~/Course/xv6-labs-2025$ |
```

图 11 Lock Lab 的 `make grade` 测试结果

测试通过表明，每 CPU 物理页空闲链表和偷页机制有效降低了 `kmem` 锁争用；读写自旋锁也正确支持多读者并发、写者独占以及写者优先等待策略。

Lab8: File system

对应源码分支: <https://github.com/JambitX11/xv6-labs-2025/tree/fs>

本实验围绕 xv6 的文件系统展开。相比前面的内存管理、trap、锁和网络实验，文件系统实验更强调磁盘上的持久化数据结构与内核运行时逻辑之间的一致性。用户看到的是 `open()`、`write()`、`read()`、`unlink()` 这样的接口；内核内部实际需要维护 inode、目录项、数据块位图、日志和路径解析等多层结构。

本 Lab 包含两个任务点。第一个任务 **Large files** 要突破 xv6 原始文件大小限制，使单个文件可以使用二级间接块，从而从最多 268 个 block 扩展到 65803 个 block。第二个任务 **Symbolic links** 要实现符号链接系统调用，并让 `open()` 在默认情况下跟随符号链接，在指定 `O_NOFOLLOW` 时打开符号链接本身。两个任务都围绕 inode 展开：前者改变 inode 中 `addrs[]` 的解释方式，后者增加一种新的 inode 类型并在路径打开过程中解析它保存的目标路径。

从整体上看，本实验训练的是在真实内核代码中把抽象概念落到具体路径上的能力。课堂上容易把文件系统理解成“树形目录”和“文件内容”，但在 xv6 中，文件内容最终要通过 `bmap()` 映射到磁盘块，删除文件时要通过 `itrunc()` 释放所有已分配块，路径名要通过 `namei()` 找到 inode，系统调用还要通过 `syscall.h`、`syscall.c`、`user.h` 和 `usys.pl` 贯通用户态与内核态。

Large files (moderate)

实验目的

本任务要求扩大 xv6 单个文件的最大容量。原始 xv6 的磁盘 inode 中有 13 个地址槽，其中前 12 个是直接块地址，最后 1 个是一级间接块地址。由于一个磁盘块大小为 `B_SIZE = 1024` 字节，一个块号占 4 字节，因此一个一级间接块可以保存 $1024 / 4 = 256$ 个数据块地址。原始最大文件块数为：

```
12 + 256 = 268 blocks
```

`bigfile` 测试要求文件能够写入 65803 个 block。题目不允许改变磁盘 inode 的总大小，因此不能简单增加 `addrs[]` 数组长度，而是要牺牲一个直接块地址槽，将 inode 地址布局改成：

```
addrs[0..10]  11 ↗ direct block
addrs[11]     1 ↗ singly-indirect block
addrs[12]     1 ↗ doubly-indirect block
```

二级间接块本身不直接指向文件数据，而是指向 256 个一级间接块，每个一级间接块再指向 256 个数据块。因此新最大文件块数为：

```
11 + 256 + 256 * 256 = 65803 blocks
```

本任务的核心目标是正确扩展文件逻辑块号到磁盘块号的映射，并保证文件删除或截断时能够释放直接块、一级间接块和二级间接块下的所有数据块。

实验步骤

实验首先修改 `kernel/fs.h` 中的文件系统格式定义。将 `NDIRECT` 从 12 改为 11，增加二级间接块容量常量，并更新 `MAXFILE`：

```
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define NDINDIRECT (NINDIRECT * NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NDINDIRECT)
```

`struct dinode` 中的 `addrs[]` 数组改为 `NDIRECT + 2` 个元素。这里的数组总长度仍然是 13，因为 `NDIRECT` 已经从 12 变为 11。这样既没有改变磁盘 `inode` 的大小，又为二级间接块保留了最后一个地址槽：

```
uint addrs[NDIRECT+2];
```

随后在 `kernel/file.h` 中同步修改内存 `inode` 的 `addrs[]` 长度。`struct dinode` 是磁盘上的 `inode` 格式，`struct inode` 是内核中的内存副本。`ilock()` 和 `iupdate()` 会在这两个结构之间复制 `addrs[]`，因此二者必须保持一致。如果只修改 `fs.h` 而不修改 `file.h`，内存中的 `inode` 视图和磁盘上的 `inode` 视图会不匹配，文件系统行为会变得不可预测。

接着修改 `kernel/fs.c` 中的 `bmap()`。`bmap()` 的参数 `bn` 是文件内部的逻辑块号，它的任务是返回对应的磁盘块号；如果该逻辑块尚未分配，写文件时还会调用 `ballocc()` 分配新磁盘块。修改后的 `bmap()` 分三段处理：

```
bn < NDIRECT
-> 直接通过 ip->addrs[bn] 找数据块

bn < NDIRECT + NINDIRECT
-> 通过 ip->addrs[NDIRECT] 找一级间接块

bn < NDIRECT + NINDIRECT + NDINDIRECT
-> 通过 ip->addrs[NDIRECT+1] 找二级间接块
```

二级间接区域的索引计算是本任务最关键的地方。进入二级间接区域后，需要先减去直接块和一级间接块覆盖的范围，然后用商和余数分别定位中间一级间接块与最终数据块：

```
bn -= NINDIRECT;  
uint i = bn / NINDIRECT;  
uint j = bn % NINDIRECT;
```

其中 *i* 表示二级间接块中的第几个一级间接块地址, *j* 表示该一级间接块中的第几个数据块地址。这样, 文件逻辑块号就被拆成两级索引:

```
ip->addrs[NDIRECT+1]  
-> doubly-indirect block  
    -> a[i] singly-indirect block  
        -> a[j] data block
```

最后修改 `itrunc()`, 使删除文件或截断文件时能够释放二级间接块。原始 `itrunc()` 只释放直接块和一级间接块。本实验增加的释放逻辑需要先读出二级间接块, 再遍历其中的每个一级间接块; 对每个存在的一级间接块, 先释放它指向的所有数据块, 再释放该一级间接块本身; 所有下级块释放完毕后, 最后释放二级间接块本身, 并将 `ip->addrs[NDIRECT+1]` 清零。

释放顺序可以概括为:

```
direct block:  
    直接释放数据块  
  
singly-indirect block:  
    释放其指向的数据块  
    释放 singly-indirect block 本身  
  
doubly-indirect block:  
    遍历其指向的每个 singly-indirect block  
    释放每个 singly-indirect block 指向的数据块  
    释放这些 singly-indirect block  
    最后释放 doubly-indirect block 本身
```

本任务实际修改文件为 `kernel/fs.h`、`kernel/file.h` 和 `kernel/fs.c`。其中 `fs.h` 和 `file.h` 负责更新 `inode` 地址数组的定义, `fs.c` 中的 `bmap()` 负责实现二级间接块寻址, `itrunc()` 负责释放新增的二级间接块结构。由于 `mkfs` 会根据 `kernel/fs.h` 构造文件系统镜像, 修改 `NDIRECT` 后必须重新生成 `fs.img`, 不能继续复用旧镜像。

实验中遇到的问题和解决方法

本任务的第一个易错点是 `inode` 地址数组的长度。题目要求不能改变磁盘 `inode` 的大小, 因此不是把 `addrs[]` 直接扩展为更多元素, 而是把 `NDIRECT` 减少为 11, 再使用 `NDIRECT+2` 保持数组总数仍为 13。这样第 12 个槽位继续表示一级间接块, 第 13 个槽位表示二级间接块。

第二个问题是 `itrunc()` 的修改容易被忽略。只修改 `bmap()` 时, `bigfile` 可能可以写出大文件,但删除或截断文件时无法释放二级间接块下的所有数据块,后续测试可能出现磁盘块泄漏或文件系统状态异常。因此本实验在实现二级间接块分配后,同步补充了 `itrunc()` 的递归式释放逻辑。

第三个问题出现在测试阶段。`bigfile` 会写入 65803 个 1KB block,且本 Lab 的 `FSSIZE` 扩大到 200000,测试运行时间明显长于前面章节。调试过程中曾出现 `make grade` 因超时终止的情况,但手工运行 `bigfile` 能输出 `bigfile done; ok, usertests -q` 也能输出 `ALL TESTS PASSED`。据此判断问题不是文件系统逻辑错误,而是本地 QEMU 与磁盘镜像运行耗时较长。最终完整等待评分脚本运行, `bigfile`、`symlinktest`、`usertests` 和 `time` 测试均通过。

实验心得

Large files 任务加深了我对 inode 地址结构的理解。文件系统上的“文件很大”并不是简单地把文件内容连续放在磁盘上,而是通过多级索引把文件内部的逻辑块号映射到真实磁盘块号。直接块适合小文件,一级间接块扩展一层,二级间接块则用一个索引块组织许多一级索引块,从而在不改变 inode 大小的前提下显著扩大文件容量。

本任务也说明,文件系统修改必须同时考虑分配和释放两条路径。`bmap()` 让文件能够长大, `itrunc()` 让文件被删除时能够把占用资源归还给文件系统。如果只关注写入路径,很容易得到一个短期能通过部分测试、但长期会泄漏磁盘块的实现。

Symbolic links (moderate)

实验目的

本任务要求为 xv6 增加符号链接。符号链接是一种特殊文件,它的内容不是普通数据,而是另一个路径名。用户打开符号链接时,内核默认应读取其中保存的目标路径,并继续打开目标文件;如果用户指定 `O_NOFOLLOW`,则不跟随目标,而是打开符号链接文件本身。

符号链接和硬链接的区别在于,硬链接让两个目录项指向同一个 inode,而符号链接拥有自己的 inode,只是在文件内容中保存目标路径字符串。因此,创建符号链接时目标路径可以不存在;只有随后 `open()` 跟随该符号链接时,目标不存在才会导致打开失败。

本任务需要把“新增文件类型”和“新增系统调用”两条路径同时打通:一方面在文件系统中增加 `T_SYMLINK` 类型,另一方面增加用户态可调用的 `symlink(target, path)` 系统调用,并修改 `open()` 的路径解析逻辑。

实验步骤

首先在 `kernel/stat.h` 中增加新的 inode 类型:

```
#define T_SYMLINK 4
```

随后在 `kernel/fcntl.h` 中增加打开标志：

```
#define O_NOFOLLOW 0x800
```

该标志用于区分两种打开行为。普通 `open()` 遇到符号链接时应跟随目标路径；带 `O_NOFOLLOW` 时则返回符号链接自身的文件描述符，测试程序可以通过 `fstat()` 检查其类型是否为 `T_SYMLINK`。

接着按照 xv6 新增系统调用的一般流程，依次修改 `kernel/syscall.h`、`kernel/syscall.c`、`user/user.h` 和 `user/usys.pl`。在 `syscall.h` 中为 `symlink` 分配新的系统调用号；在 `syscall.c` 中声明 `sys_symlink()` 并放入 `syscalls[]` 表；在 `user/user.h` 中声明用户态函数原型；在 `user/usys.pl` 中加入 `entry("symlink")` 以生成用户态汇编桩。这样用户程序调用 `symlink()` 时，才能通过 `ecall` 进入内核中的 `sys_symlink()`。

`sys_symlink()` 实现在 `kernel/sysfile.c` 中。它从用户参数中读取目标路径 `target` 和新链接路径 `path`，通过已有的 `create()` 创建一个类型为 `T_SYMLINK` 的 `inode`，然后将 `target` 字符串写入该 `inode` 的文件内容中：

```
symlink(target, path)
-> 创建 path 对应的 T_SYMLINK inode
-> 将 target 字符串写入这个 inode
-> 返回 0
```

这里需要注意 `create()` 在 `sysfile.c` 中是 `static` 函数。实验中曾因将 `sys_symlink()` 写在 `create()` 前面，导致编译器报出 `implicit declaration of function 'create'` 和返回类型冲突。解决方法是将 `sys_symlink()` 放在 `create()` 定义之后、`sys_open()` 之前，或者提前声明 `create()`。本实验采用前一种方式，使代码顺序也更符合“先有创建 helper，再实现具体创建类系统调用”的逻辑。

随后修改 `sys_open()`。原始 `open()` 在非 `O_CREATE` 情况下通过 `namei(path)` 找到 `inode`，然后直接打开。加入符号链接后，需要在 `inode` 加锁后判断其类型：

```
如果 ip->type == T_SYMLINK 且未设置 O_NOFOLLOW:
    读出符号链接文件中保存的目标路径
    释放当前符号链接 inode
    使用 namei(target) 查找目标 inode
    继续判断目标是否仍然是符号链接
```

为了避免符号链接环导致无限循环，例如 `a -> b`、`b -> a`，`open()` 中设置了最大跟随深度。若超过限制，则认为可能存在循环并返回错误。这样既能支持多层符号链接，也能避免内核在错误路径结构中无限解析。

最后修改 Makefile, 在 LAB=fs 时将 user/symlinktest.c 编译进文件系统镜像, 使 xv6 shell 中可以运行 symlinktest。本任务实际修改文件包括 kernel/stat.h、kernel/fcntl.h、kernel/syscall.h、kernel/syscall.c、kernel/sysfile.c、user/user.h、user/usys.pl 和 Makefile。

实验中遇到的问题和解决方法

本任务首先遇到的是 `create()` 的声明顺序问题。`sys_symlink()` 需要复用 `create(path, T_SYMLINK, 0, 0)` 创建符号链接 inode, 但 `create()` 是 kernel/sysfile.c 内部的 static 函数。如果 `sys_symlink()` 写在 `create()` 定义之前, C 编译器会在第一次看到 `create()` 调用时将其视为隐式声明的 `int create()`, 随后遇到真实定义 `static struct inode *create(...)` 时就发生类型冲突。最终通过调整函数顺序解决该编译错误。

第二个问题是符号链接的打开语义。`symlink()` 创建时不能要求目标文件已经存在, 因为符号链接保存的是路径字符串, 而不是目标 inode 的引用。目标路径不存在时, 创建符号链接仍应成功; 只有在 `open()` 跟随该符号链接时, `namei(target)` 找不到目标才返回失败。本实验在 `sys_symlink()` 中只负责创建 `T_SYMLINK` inode 和保存目标路径, 不对目标路径做存在性检查。

第三个问题是符号链接循环。若 `open()` 只是简单地不断跟随 `T_SYMLINK`, 遇到 `a -> b`、`b -> a` 这样的结构就会无限循环。实验通过设置最大解析层数解决这一问题; 当跟随次数超过上限时, `open()` 返回 `-1`, 与测试中对循环链接的预期一致。

实验心得

Symbolic links 任务展示了文件系统路径解析的另一面。硬链接直接增加 inode 的目录入口引用, 而符号链接通过一个独立 inode 保存路径字符串。也就是说, 符号链接不是“另一个文件名直接指向同一份文件内容”, 而是“打开时请再按这个字符串重新走一次路径查找”。理解这一点后, 为什么 `symlink()` 不要求目标存在、为什么 `unlink()` 应删除符号链接本身、为什么 `open()` 需要专门处理跟随逻辑, 都变得更清晰。

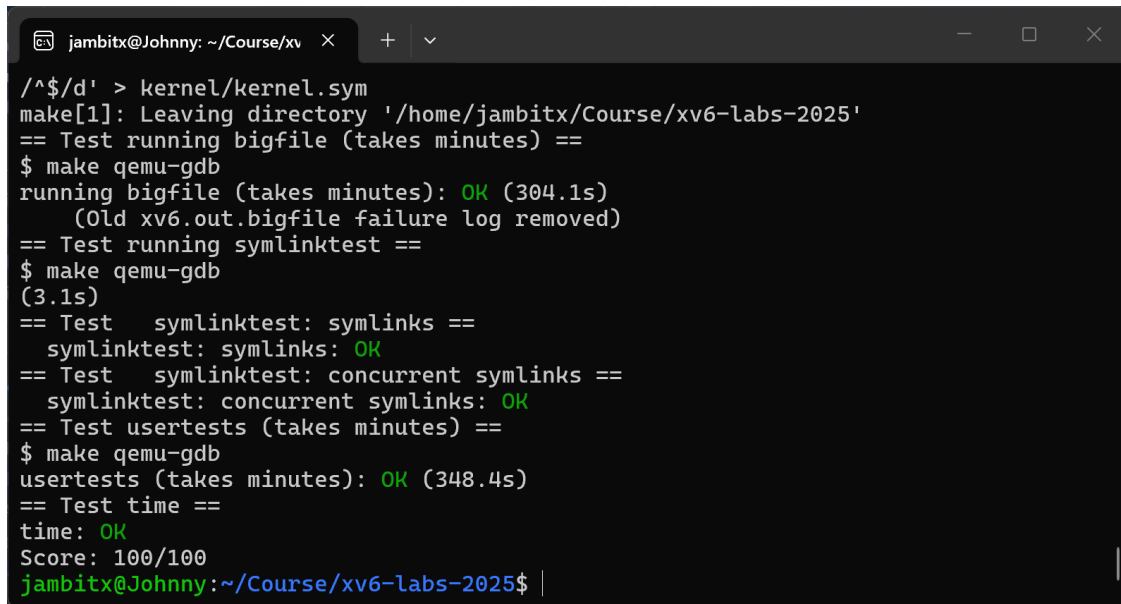
本任务也再次复习了 xv6 新增系统调用的完整链路。仅在内核中写一个 `sys_symlink()` 不够, 还必须分配系统调用号、注册系统调用表、声明用户态原型、生成用户态汇编入口, 并把测试程序加入文件系统镜像。这个流程和前面 `syscall lab` 中的经验一致, 但本任务进一步把系统调用和文件系统内部 helper 结合起来。

实验结果

完成本 Lab 后, 在 fs 分支运行:

```
$ make grade
```


最终 `bigfile`、`symlinktest`、`usertests` 和 `time` 测试均通过，得分为满分。由于 `bigfile` 需要写入 65803 个 block，`usertests` 也会受到较大的 `FSSIZE` 影响，本 Lab 的评分运行时间明显长于前面章节。测试结果如下图所示。



```
jambitx@Johnny: ~/Course/xv6 X + v
/^$/d' > kernel/kernel.sym
make[1]: Leaving directory '/home/jambitx/Course/xv6-labs-2025'
== Test running bigfile (takes minutes) ==
$ make qemu-gdb
running bigfile (takes minutes): OK (304.1s)
(Old xv6.out.bigfile failure log removed)
== Test running symlinktest ==
$ make qemu-gdb
(3.1s)
== Test    symlinktest: symlinks ==
    symlinktest: symlinks: OK
== Test    symlinktest: concurrent symlinks ==
    symlinktest: concurrent symlinks: OK
== Test usertests (takes minutes) ==
$ make qemu-gdb
usertests (takes minutes): OK (348.4s)
== Test time ==
time: OK
Score: 100/100
jambitx@Johnny: ~/Course/xv6-labs-2025$ |
```

图 12 File system Lab 的 make grade 测试结果

测试通过表明，二级间接块扩展、文件截断释放、符号链接创建、符号链接跟随、`O_NOFOLLOW` 语义以及文件系统回归测试均符合实验要求。

Lab9: mmap

对应源码分支：<https://github.com/JambitX11/xv6-labs-2025/tree/mmap>

本实验实现 `xv6` 中面向文件的 `mmap()` 和 `munmap()` 机制。`mmap` 的作用是把文件的一段内容映射到进程的虚拟地址空间中，使用户程序可以像访问普通内存一样访问文件内容；`munmap` 则负责解除映射，并在必要时把用户对映射内存的修改写回文件。与普通 `read()` 和 `write()` 相比，`mmap()` 的特殊之处在于它把文件系统和虚拟内存系统连接在一起：文件内容不再只通过文件描述符顺序读写，而是通过 `page fault` 懒加载到用户页表中。

这个实验虽然在题目上只有一个任务点，但实际涉及多条内核路径。首先，内核需要新增 `mmap` 和 `munmap` 两个系统调用入口；其次，每个进程需要记录自己的文件映射区域，即 `VMA`；再次，用户访问尚未加载的映射页时，内核需要在 `page fault` 路径中分配物理页、读取文件内容并建立页表映射；最后，`munmap()`、`exit()` 和 `fork()` 都必须正确处理 `VMA` 的生命周期。也就是说，本实验的核心不是“把文件一次性读入内存”，而是建立一种延迟的、按页加载的文件到虚拟地址空间的关系。

Memory-mapped files (hard)

实验目的

本任务要求在 xv6 中实现足以通过 `mmaptest` 的内存映射文件功能。用户调用：

```
mmap(addr, len, prot, flags, fd, offset)
```

时，本实验只需要支持题目限定的子集：`addr` 和 `offset` 可以假定为 0，`prot` 主要包含 `PROT_READ`、`PROT_WRITE` 和 `PROT_EXEC`，`flags` 主要包含 `MAP_SHARED` 与 `MAP_PRIVATE`。`MAP_SHARED` 表示对映射内存的修改需要写回文件，`MAP_PRIVATE` 表示修改只影响当前进程，不写回原文件。

实验要求 `mmap()` 本身采用 `lazy` 策略，即系统调用返回时不分配物理页，也不读取文件。内核只在进程真正访问映射地址并触发 `page fault` 时，才分配一个物理页，从文件中读入对应页的数据，并将该页映射到用户页表中。这样的设计保证映射大文件时不会因为 `mmap()` 一次性读入所有内容而变慢，也允许映射文件大小超过可用物理内存。

本实验的正确性目标包括：文件内容能够通过映射地址读取；`MAP_PRIVATE` 的修改不写回文件；`MAP_SHARED` 的修改在 `munmap()` 或进程退出时写回文件；只读映射不允许写入；`munmap()` 后继续访问应触发异常并杀死进程；`fork()` 后子进程能够继承父进程的 VMA，并在访问时独立懒加载映射页。

实验步骤

实验首先补齐系统调用入口。在 `kernel/syscall.h` 中为 `mmap` 和 `munmap` 分配系统调用号，在 `kernel/syscall.c` 中声明 `sys_mmap()` 和 `sys_munmap()` 并加入 `syscalls[]` 表，在 `user/user.h` 中声明用户态函数原型，在 `user/usys.pl` 中加入对应的系统调用桩。这样，`user/mmaptest.c` 才能从用户态通过 `ecall` 进入内核实现。`kernel/fcntl.h` 已在 `LAB_MMAP` 条件下给出 `PROT_READ`、`PROT_WRITE`、`PROT_EXEC`、`MAP_SHARED` 和 `MAP_PRIVATE` 等常量，后续实现围绕这些标志检查权限。

随后在 `kernel/proc.h` 中增加 VMA 结构。VMA 是 `virtual memory area` 的缩写，用来描述一个进程地址空间中的一段连续映射区域。本实验采用固定大小数组，因为 xv6 内核没有通用的可变长度内核分配器，题目也提示 16 个 VMA 足够通过测试。结构中记录是否被使用、起始虚拟地址、映射长度、权限、映射标志、文件偏移和对应的 `struct file *`：

```
#define NVMA 16

struct vma {
    int used;
    uint64 addr;
    uint64 len;
    int prot;
    int flags;
```

```
uint64 offset;
struct file *file;
};
```

每个 struct proc 中增加:

```
struct vma vmas[NVMA];
```

kernel/proc.c 中相应修改 allocproc() 和 freeproc(), 在进程结构被分配或回收时初始化 VMA 表, 避免复用 struct proc 时残留旧映射信息。

sys_mmap() 实现在 kernel/sysfile.c 中。由于这里已有 argfd(), 可以方便地从用户传入的文件描述符取得 struct file *。sys_mmap() 的主要工作不是读文件, 而是验证参数并登记 VMA。它检查 len 是否有效、addr 和 offset 是否符合实验假设、映射权限是否与文件打开权限兼容。例如, 如果用户要求 MAP_SHARED | PROT_WRITE, 则文件必须以可写方式打开; 否则用户通过共享映射写入后无法合法写回文件。

在选择映射地址时, 实验从 TRAPFRAME 下方向低地址分配 VMA 区域。普通用户程序的 text、data、heap 从低地址向上增长, 而 TRAPFRAME 与 TRAMPOLINE 位于用户虚拟地址空间最高处。将 mmap 区域放在高地址并向低地址分配, 可以减少与普通堆空间冲突的可能。mmap() 成功后调用 filedup() 增加文件引用计数, 因为用户可能在 mmap() 之后立即 close(fd); VMA 必须独立持有文件引用, 映射才能继续有效。

页的真正加载发生在 kernel/vm.c 的 vmfault() 中。用户访问 mmap 区域时, RISC-V 触发 load page fault 或 store page fault, kernel/trap.c 中的 usertrap() 调用 vmfault()。修改后的 vmfault() 先检查地址是否小于 MAXVA, 然后判断该地址是否属于普通 lazy sbrk 区域或某个 VMA。若地址位于 VMA 中, 就按照访问类型检查权限: 读 fault 要求 PROT_READ, 写 fault 要求 PROT_WRITE。权限合法时分配物理页, 将文件中对应偏移的数据读入该页, 再根据 prot 设置 PTE_R、PTE_W、PTE_X 和 PTE_U 权限并调用 mappages() 建立映射。

文件偏移由 VMA 起点和 fault 地址共同决定:

```
file offset = vma.offset + (fault_va - vma.addr)
```

因此同一个文件映射中的不同虚拟页会从文件的不同位置懒加载。若映射长度超过文件实际大小, readi() 只会读到文件已有部分, 物理页剩余部分由于 kalloc() 后执行了 memset(), 自然保持为零。这正好符合测试中 1.5 页文件映射为 2 页时, 最后半页应读到 0 的要求。

sys_munmap() 负责解除映射。实现中先根据用户传入地址找到对应 VMA, 再按页遍历需要解除的范围。由于 mmap 是懒加载, 有些页可能从未访问过, 页表中没有有效 PTE; 这种情况应直接跳过, 不能 panic。对于已经实际映射的页, 如果该 VMA 是 MAP_SHARED, 则在解除映射前将对应页内容写回文件。写回时按页单独使用 begin_op() 和 end_op(),

避免一个日志事务覆盖太多数据块；同时写回长度不能超过文件当前大小，防止把测试中原本只有 1.5 页的文件错误扩展为 2 页。完成写回后调用 `uvmunmap()` 取消页表映射并释放物理页。

`munmap()` 还需要维护 VMA 的范围。题目保证不会在 VMA 中间“打洞”，只会解除开头、结尾或整个区域。因此实现分三种情况：若解除整个 VMA，则调用 `fileclose()` 释放文件引用并清空 VMA；若解除开头部分，则将 `vma.addr` 向后移动，并同步增加 `vma.offset`；若解除结尾部分，则缩短 `vma.len`。这样，剩余区域仍然能在后续 page fault 中找到正确的文件偏移。

进程退出和 `fork` 也需要处理 `mmap`。`kernel/proc.c` 的 `kexit()` 中，在关闭普通文件描述符之前遍历所有 VMA，并按 `munmap()` 的语义写回与释放映射区域。这样即使用户没有显式调用 `munmap()`，`MAP_SHARED` 的修改也不会丢失。`kfork()` 中复制父进程的 VMA 表给子进程，并对每个有效 VMA 调用 `filedup()`。本实验不要求父子进程共享同一物理页，因此 `fork` 时只复制 VMA 元信息即可；子进程第一次访问映射地址时，会在自己的 page fault 路径中独立分配和读入物理页。

此外，`copyin()` 和 `copyout()` 也需要与懒加载配合。内核从用户地址读取数据时，如果该地址属于尚未加载的 `mmap` 区域，应允许 `copyin()` 触发 `vmfault()` 来补页；内核向用户地址写数据时，`copyout()` 也可能需要补出可写映射页。这样系统调用读写用户缓冲区时不会因为 `mmap` 页尚未物理分配而错误失败。

本任务实际修改文件包括 `kernel/syscall.h`、`kernel/syscall.c`、`user/user.h`、`user/usys.pl`、`kernel/proc.h`、`kernel/proc.c`、`kernel/sysfile.c`、`kernel/vm.c`、`kernel/defs.h` 和 `Makefile`。其中系统调用相关文件负责用户态到内核态的入口，`proc.h` 和 `proc.c` 负责 VMA 生命周期，`sysfile.c` 负责 `mmap()` 与 `munmap()` 语义，`vm.c` 负责 page fault 懒加载和用户地址复制路径，`defs.h` 用于暴露进程退出时调用的 `mmap` 清理函数。

实验中遇到的问题和解决方法

本实验首先遇到的是头文件依赖问题。在 `kernel/vm.c` 中访问 `struct file` 和 `vma->file->ip` 时，需要包含 `file.h`；但 `file.h` 中的 `struct inode` 包含 `struct sleeplock` `lock`，因此必须先包含 `sleeplock.h`，同时 `PROT_READ`、`PROT_WRITE` 和 `PROT_EXEC` 定义在 `fcntl.h` 中。最终通过在 `vm.c` 中按顺序包含 `fs.h`、`sleeplock.h`、`file.h` 和 `fcntl.h` 解决了 `field 'lock' has incomplete type` 以及 `PROT_READ undeclared` 等编译错误。

第二个问题出现在 `kernel/sysfile.c`。`sys_mmap()` 选择从 `TRAPFRAME` 下方分配 `mmap` 虚拟地址，因此需要包含 `memlayout.h`。如果没有包含该头文件，编译器会报出 `TRAPFRAME undeclared`。同时，调试过程中曾定义了未被使用的 `find_vma()` 辅助函数，在 `xv6` 的 `-Werror` 编译设置下，未使用的 `static` 函数也会导致编译失败。解决方法是补充 `#include "memlayout.h"`，并删除未使用的辅助函数或确保它被实际调用。

第三个问题是 `usertests` 的 `copyin` 测试触发 `panic: walk`。原因是 `copyin()` 在访问坏用户地址时调用了 `vmfault()`，而 `vmfault()` 入口没有先检查 `va >= MAXVA`，后

续 `ismapped()` 调用 `walk()` 时触发了 `walk()` 内部的非法地址 panic。正确行为应该是系统调用返回错误，而不是内核 panic。最终在 `vmfault()` 开头加入地址范围检查，并在 `ismapped()` 中也补充保护，使非法用户地址能够被安全拒绝，`usertests` 中的 `copyin` 测试随即通过。

第四个需要特别注意的问题是 `munmap()` 写回长度。`mmaptest` 中构造的文件长度为 1.5 页，却映射了 2 页。如果对第二页写回完整 `PGSIZE`，会把文件错误扩展到两页。实验通过读取 `inode` 的 `size` 并限制每页写回长度，保证只把文件原本存在的范围写回。未映射或未访问过的页也必须跳过，因为 lazy loading 使 VMA 中的部分虚拟页可能从未建立过 PTE。

最后，`mmaptest` 中“`munmap` 后访问”和“写只读映射”会故意制造用户态 page fault。测试输出中出现 `usertrap(): unexpected scause` 并不一定表示实验失败；关键是子进程应被杀死，父进程随后确认状态并输出对应 OK。这一现象说明权限位和解除映射后的页表状态都按预期工作。

实验心得

`mmap` 实验把前面多个 lab 的知识联系在了一起。它既需要 `syscall` lab 中新增系统调用的完整链路，也需要 lazy allocation 和 COW lab 中对 page fault 的理解，还需要 file system lab 中对 `readi()`、`writeri()`、`inode` 锁和日志事务的认识。只有把这些机制串成一条数据流，才能理解为什么 `mmap()` 本身不读文件，而是在用户第一次访问映射地址时才真正分配和读取。

本实验也让我更清楚地区分了“虚拟地址区域”和“实际物理页”。VMA 只是进程地址空间中一段承诺：这段地址如果被访问，应当来自某个文件。它并不等于已经分配好的物理内存。真正的物理页只有在 page fault 发生后才出现，并且可以在 `munmap()` 时释放。这种设计使操作系统能够用较低成本支持大文件映射，也解释了为什么现代系统普遍采用按需分页。

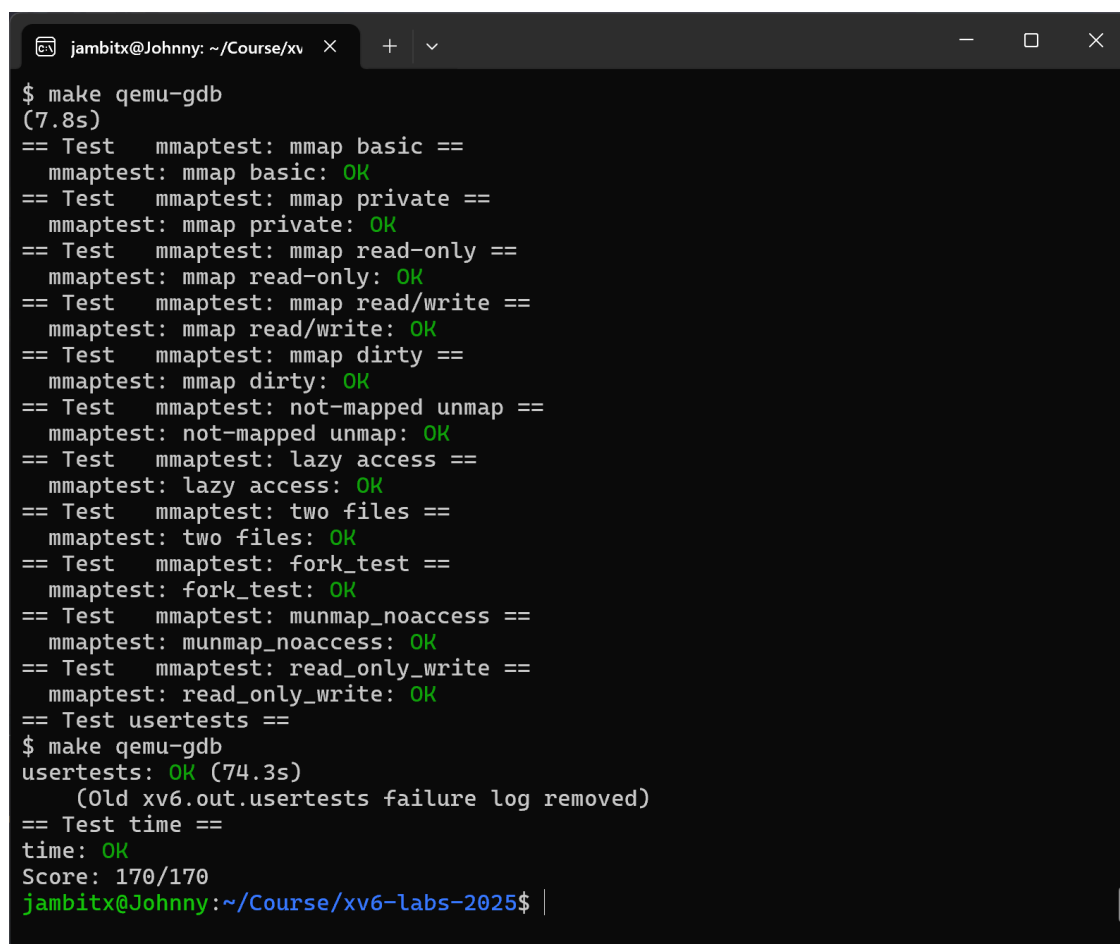
此外，`mmap` 的难点不在单个函数，而在生命周期的闭环。`mmap()` 要持有文件引用，page fault 要读入内容，`munmap()` 要写回和释放，`exit()` 要代替用户清理遗留映射，`fork()` 要复制 VMA 并维护引用计数。任何一个环节遗漏，都会表现为文件修改丢失、引用泄漏、页表释放 panic 或子进程访问失败。通过本实验可以更直观地认识到，内核功能往往不是孤立实现的接口，而是一组跨模块状态在多个路径上的一致维护。

实验结果

完成本 Lab 后，在 `mmap` 分支运行：

```
$ make grade
```

最终 mmaptest 中的 basic、private、read-only、read/write、dirty、not-mapped unmap、lazy access、two files、fork、munmap_noaccess 和 read_only_write 测试均通过，usertests 和 time 测试也通过，得分为满分。测试结果如下图所示。



```
jambitx@Johnny: ~/Course/xv  X + v
$ make qemu-gdb
(7.8s)
== Test  mmaptest: mmap basic ==
mmaptest: mmap basic: OK
== Test  mmaptest: mmap private ==
mmaptest: mmap private: OK
== Test  mmaptest: mmap read-only ==
mmaptest: mmap read-only: OK
== Test  mmaptest: mmap read/write ==
mmaptest: mmap read/write: OK
== Test  mmaptest: mmap dirty ==
mmaptest: mmap dirty: OK
== Test  mmaptest: not-mapped unmap ==
mmaptest: not-mapped unmap: OK
== Test  mmaptest: lazy access ==
mmaptest: lazy access: OK
== Test  mmaptest: two files ==
mmaptest: two files: OK
== Test  mmaptest: fork_test ==
mmaptest: fork_test: OK
== Test  mmaptest: munmap_noaccess ==
mmaptest: munmap_noaccess: OK
== Test  mmaptest: read_only_write ==
mmaptest: read_only_write: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (74.3s)
(Old xv6.out.usertests failure log removed)
== Test time ==
time: OK
Score: 170/170
jambitx@Johnny:~/Course/xv6-labs-2025$ |
```

图 13 mmap Lab 的 make grade 测试结果

测试通过表明，VMA 登记、文件映射懒加载、共享映射写回、私有映射隔离、部分解除映射、退出清理、fork 继承以及非法访问保护等关键路径均符合实验要求。